

MyPetTrip

AI 기반 반려동물 동반 장소 추천 서비스

트랙: 산학

팀번호: 25

팀명: return 0;

팀원:

- 2076260 유예나 (컴퓨터공학과) – 팀장 / DB·Backend / Mobile App Dev
- 2017003 계예진 (컴퓨터공학과) – PM / UI·UX / Mobile App Dev
- 2076133 민성원 (컴퓨터공학과) – AI / Mobile App Dev

팀 지도교수: 심재형 교수님

1. Team Info

1.1 과제명

My Pet Trip - AI 기반 반려동물 동반 장소 추천 서비스

1.2 팀 정보

본 프로젝트는 이화여자대학교 컴퓨터공학과 캡스톤디자인 과제로 수행되었으며, 팀 번호 25번, 팀명 return 0;인 팀의 보고서이다. 본 팀은 반려동물 동반 과정에서 발생하는 정보 탐색의 어려움과 비효율성을 해결하는 것을 목표로 해당 프로젝트를 기획하였다.

1.3 팀 구성원

유예나(팀장)는 전체 시스템 구조 설계와 데이터베이스 설계, Firebase 기반 백엔드 구성을 담당하였으며, 민성원은 AI 기반 여행 상담 로직 및 백엔드 연동 기능 구현을 맡았다. 계예진은 사용자 경험을 고려한 UI·UX 설계와 Flutter 기반 모바일 애플리케이션 개발을 담당하였다.

각 팀원은 역할을 분담하되, 서비스 기획 및 기능 정의 과정에서는 공동으로 참여하여 전체적인 방향성을 함께 설정하였다.

2. Project Summary

2.1 문제 정의

대한민국의 반려동물 양육 인구는 매년 증가하여 2024년 기준 등록 반려동물 수가 약 349만 마리에 달하며, 반려동물을 가족 구성원으로 여기는 문화가 빠르게 확산되고 있다. 제주관광공사의 설문 조사에 따르면 응답자의 81.6%가 반려동물과 함께 여행한다고 답변할 만큼 반려동물 동반 여행 수요 역시 높아지고 있다.

그러나 반려동물 친화 시설의 증가 속도와 달리, 반려인이 실제로 여행을 준비하는 과정에서 겪는 불편함은 해소되지 않고 있다. 시설이 '반려동물 동반 가능'으로 표기되어 있더라도, 실제 현장에서는 견종·체중 제한, 실내·실외 구역 구분, 이동 가방·목줄 규정 등 세부 조건이 존재하는 경우가 많아 보호자가 이를 사전에 정확히 파악하기 어렵기 때문이다. 이로 인해 사용자는 여행 준비 과정에서 많은 시간과 노력을 소비할 뿐 아니라, 현장 방문 후 입장이 거부되는 실패 경험까지 겪게 된다.

본 프로젝트는 반려견 동반 여행을 주요 대상으로 하며, 여행 준비 과정에서 발생하는 핵심 문제를 다음 **세 가지**로 정의한다.

【문제 1】 반려동물 동반 정보의 파편화 및 신뢰성 부족

반려동물 동반 정보가 지도 서비스, 포털 검색, 블로그, 커뮤니티, SNS 등 여러 플랫폼에 분산되어 있어, 신뢰도 높은 정보를 한 곳에서 수집하기 어렵다. 각 시설의 반려동물 정책(견종·체중 제한, 구역 구분, 목줄 규정 등)이 명확히 제시되지 않거나, 실제 운영 조건과 불일치하는 경우가 많아 방문 실패 경험이 반복된다. 또한 정보의 최신성을 검증할 수 있는 체계가 부족하여 과거의 잘못된 정보를 신뢰하고 방문하는 상황이 발생한다.

【문제 2】 맞춤형 추천 기능의 부재

기존 서비스는 단순한 장소 목록 제공 수준에 머물러 있어, 반려동물의 크기뿐만 아니라 활동성, 사회성 등 개별 성향과 보호자의 선호를 복합적으로 반영한 맞춤형 추천 기능이 부족하다. 예를 들어, 활동성이 낮은 노령견이나 소음에 민감한 반려동물을 동반한 사용자의 경우 단순 키워드 검색만으로는 적합한 장소를 찾기 어렵다. 결과적으로 사용자는 원하는 장소보다 출입 제한이 적은 곳을 우선 선택하게 되어 여행의 전반적인 만족도가 저하된다.

위 두 가지 문제는 독립적이지 않으며, 상호 연결되어 있다. 정보가 파편화되어 있기 때문에 맞춤형 추천의 기반이 되는 정형화된 장소 데이터가 존재하지 않고, 맞춤형 추천 기능이 없기 때문에 사용자는 수동으로 여러 플랫폼을 탐색하는 반복 작업을 수행할 수밖에 없다. 이 악순환의 결과로 반려인은 여행 준비 과정에서 과도한 시간을 소비하게 된다. 따라서 본 프로젝트는 이러한 비효율을 개선하기 위해 공공데이터를 기반으로 반려동물 동반 정보를 체계적으로 구조화하고, 신뢰도 높은 장소 DB를 구축하고자 한다. 이는 사용자 및 반려동물 특성을 함께 고려한 맞춤형 탐색과 추천을 지원하는 서비스의 필요성에서 출발한다.

2.2 기존 연구와의 비교

2.2.1 기존 서비스 현황 분석

현재 반려동물 동반 여행 관련 서비스는 크게 세 가지 유형으로 분류할 수 있다.

(1) 범용 지도·포털 서비스 (네이버 지도, 카카오맵)

- 반려동물 동반 가능 여부를 장소 정보에 일부 포함하지만, 사용자 블로그 포스팅과 리뷰에 의존하여 정보의 정확성과 최신성이 보장되지 않는다.
- 체중 제한, 견종 제한, 실내·실외 구역 구분 등 세부 정책 정보는 거의 제공되지 않으며, 반려동물 특성을 반영한 맞춤형 추천 기능은 존재하지 않는다.

(2) 반려동물 동반 여행 전문 앱 (반**할)

- 반려동물 동반 숙소·카페·식당 등을 통합하여 제공하고, 야놀자와 제휴해 전국 1,100여 개 동반 숙소의 실시간 예약 기능을 지원하는 국내 가장 유사한 서비스이다.
- 대형견 가능 여부, 독채 여부 등 일부 조건 필터를 제공하나, 반려동물의 성향(소음 민감도, 사회성, 활동성 등)이나 건강 상태를 고려한 AI 기반 맞춤형 추천 기능은 없다.
- 정보 최신성은 사용자 후기에 의존하며, 데이터 검증 체계가 부재하여 오래된 정보가 노출되는 문제가 발생한다.

(3) 반려동물 동반 장소 검색 서비스 (와*, 하*독 등)

- 반려동물과 함께 입장 가능한 장소를 지도 기반으로 검색하고, 업종별·조건별 정렬 기능을 제공하는 서비스이다.
- 장소 목록 탐색 및 찜 기능 중심으로 구성되어 있으며, 개별 반려동물의 특성과 보호자 선호를 반영한 개인화 추천 기능은 지원하지 않는다.
- 데이터 수집과 검증이 수동 또는 사용자 제보에 의존하여 정보의 상세도와 신뢰성 확보에 한계가 있다.

2.2.2 기존 서비스 대비 비교 분석

기존 서비스들이 공통으로 해결하지 못하는 문제는 정확히 두 가지다. 첫째, 반려동물 동반 정보의 신뢰성이 담보되지 않으며, 둘째, 개별 반려동물의 특성을 반영한 맞춤 추천이 존재하지 않는다. MyPetTrip은 이 두 축을 기술적으로 해결하는 것을 핵심 목표로 한다. 아래 표는 이 두 가지 관점에서 기존 서비스들과의 차별점을 정리한 것이다.

비교 항목	범용 지도 서비스 (네이버·카카오맵)	반**할 (동반 여행 앱)	와* (동반 장소 검색)	MyPetTrip (제안 시스템)
[신뢰성] 세부 정책 정보 (체중·견종·구역)	미제공	일부 제공	일부 제공 (조건 필터 중심)	GPT 기반 자동 추출로 세부 정책까지 표시
[신뢰성] 정보 출처 및 검증 방식	사용자 리뷰 의존 검증 체계 없음	사용자 후기 의존 검증 체계 없음	사용자 제보 의존 검증 체계 없음	공공데이터 + 웹 비정형 데이터 교차검증 후 DB 저장
[신뢰성] 정보 최신성 관리	수동 업데이트 (사실상 방치)	수동 업데이트 (사실상 방치)	수동 업데이트 (사실상 방치)	이상 감지 시스템 + 정보 최신성 갱신 파이프라인
[맞춤 추천] 반려동물 특성 반영 여부	없음	없음	없음	크기·성향 등 프로필 기반 컨텍스트 주입
[맞춤 추천] 추천 방식	키워드 검색 수동 탐색	카테고리/조건 필터 직접 선택	카테고리/조건 필터 직접 선택	자연어 챗봇 기반 RAG 구조 맞춤 추천

2.2.3 관련 기술 연구와의 비교

MyPetTrip의 기술적 접근 방식은 다음의 연구 흐름과 차별화되며, 두 핵심 축인 '신뢰성'과 '반려동물 특성 기반 추천'을 중심으로 설계된다.

(1) 정보 신뢰성 확보: 웹 기반 비정형 데이터 수집 및 LLM 기반 정보 추출

공공데이터 API는 장소의 존재 여부는 제공하지만 '15kg 이하만 가능', '테라스만 동반 가능' 등의 실질적 운영 정책은 포함하지 않는다. MyPetTrip은 공식 웹사이트 및 리뷰 플랫폼, 블로그나 후기 등 웹 기반 비정형 텍스트 데이터를 수집하고, GPT API 기반 정보 추출 파이프라인을 통해 체중 제한·출입 구역·동반 조건 등 세부 정책을 정형 데이터로 변환하여 DB에 저장한다. 이는 기존 서비스들이 사용자 신고에만 의존하거나 사실상 업데이트를 방치하던 정보 수집 방식을 공학적으로 해결한 것이다.

(2) 정보 신뢰성 유지: 이상 감지 및 갱신 파이프라인

반려동물 동반 정책은 계절·운영 방침 변경 등으로 수시로 변경된다. MyPetTrip은 이를 반영하기 위해 정기적 재크롤링과 GPT 재판별을 통해 DB를 갱신하고, 사용자 신고 누적·트래픽 급감·부정 키워드 증가 등 다차원 신호를 기반으로 정보 오류를 감지하는 이상 감지 시스템을 결합한다. 이는 실세계 동적 정보를 지속적으로 최신 상태로 유지하는 데이터 엔지니어링 문제에 대한 공학적 접근이다.

(3) 반려동물 특성 기반 추천: 도메인 특화 RAG 구조 (Retrieval-Augmented Generation)

기존 LLM 기반 챗봇은 학습 데이터에 기반한 일반 지식만을 활용하므로, 특정 도메인의 최신·정확한 정보를 보장하지 못한다. 또한, 특정 장소에 대한 정보를 생성할 때 학습 데이터에 없는 사실을 지어내는 환각(Hallucination) 문제가 발생한다. MyPetTrip은 GPT를 통해 추천 생성 시 반드시 자체 검증 DB를 조회하도록 강제하는 RAG 구조를 채택하여, 환각 없이 검증된 DB 기반 추천을 생성한다. 또한 여기에 등록된 반려동물 프로필(크기·성향·활동성 등)을 시스템 컨텍스트로 주입함으로써, 동일한 질문이라도 각 반려동물의 특성에 맞는 다른 장소가 추천되는 진정한 의미의 맞춤형 추천을 실현한다

2.3 제안 내용

2.3.1 서비스 개요

MyPetTrip은 공공데이터 기반의 반려동물 동반 장소 정보를 수집·정제·표준화하고, GPT 기반 챗봇을 통해 사용자가 자연어로 반려동물 동반 여행을 탐색하고 추천받을 수 있는 반려동물 동반 **장소 추천** 여행 애플리케이션이다. MyPetTrip은 반려견 동반 여행을 주요 서비스 대상으로 하며, 본 서비스의 핵심 가치는 두 가지로 요약된다.

첫째, 기존 플랫폼이 해결하지 못했던 반려견 동반 정보의 신뢰성을 **여러 방식의 데이터 파이프라인으로 데이터 수집·정제·검증 파이프라인을 통해** 확보한다. 둘째, 개별 반려견의 특성을 실질적으로 반영하는 맞춤형 추천을 GPT 기반 챗봇을 통해 실현한다. 본 서비스는 이 두 가지 문제를 동시에 해결하는 것을 핵심 설계 방향으로 삼는다.

2.3.2 핵심 제안 기술

2.3.2 Target customer

대상 고객은 반려견과 함께 카페, 숙소, 관광지 등을 방문하려는 반려견 보호자이며, 특히 기존 지도 서비스나 블로그 검색만으로는 정확한 동반 가능조건을 확인하기 어려운 사용자를 주요 타겟으로 한다.

또한 체중 제한, 견종 제한, 실내 이용 가능 여부 등 세부 조건을 사전에 확인해야 하는 소형견·중대형견 보호자와, 맹견, 노견 등 개별 특성을 고려한 장소 추천이 필요한 사용자를 핵심 고객으로 설정한다. 따라서 MyPetTrip은 단순히 “반려동물 동반 가능 장소”를 찾는 사용자가 아니라, 자신의 반려견 조건에 실제로 적합한 장소를 빠르고 신뢰성 있게 찾고자 하는 보호자를 주요 대상으로 한다.

2.3.3 핵심 제안 기술

본 프로젝트는 ‘신뢰성 확보’와 ‘반려견 특성 기반 추천’이라는 두 핵심 문제를 해결하기 위해 다음 세 가지 기술 요소를 제안한다.

【기술 1】 다중 출처 데이터 통합 파이프라인 — 신뢰성의 기반 구축

기존 플랫폼이 단일 출처(사용자 리뷰 또는 공공데이터)에 의존하는 구조적 한계를 극복하기 위해, MyPetTrip은 두 종류의 데이터를 통합하는 3단계 파이프라인을 설계한다.

1. 공공데이터 수집 (TourAPI → Firestore)

한국관광공사 TourAPI 4.0의 반려동물 동반 여행 카테고리를 호출하여 전국 단위 장소의 기초 데이터(장소명, 주소, 좌표, 전화번호 등)를 자동 수집하고 Firestore에 저장한다.

2. 블로그·후기 크롤링을 통한 세부 정보 보완

네이버 검색 API와 BeautifulSoup을 활용하여 각 장소명으로 공식 웹사이트 및 리뷰 플랫폼, 블로그 포스팅 및 방문 후기를 수집한다. 수집 대상은 장소의 실제 운영 정책(체중 제한, 견종 제한, 실내·실외 구역 구분, 목줄·이동가방 규정 등)이 언급되는 텍스트이며, 공공데이터에 포함되지 않은 세부 조건 정보를 확보하는 것이 목적이다.

3. GPT 기반 반려견 동반 가능 여부 자동 판별 (AI 정보 추출)

수집된 후기·블로그 텍스트를 OpenAI GPT API에 입력하고, 구조화된 프롬프트(예: '아래 텍스트에서 반려견 동반 가능 여부, 체중 제한, 출입 가능 구역을 JSON 형태로 추출하라')를 통해 비정형 텍스트에서 정형화된 정책 데이터를 자동 추출한다. 추출된 결과는 Firestore의 해당 장소 문서에 Upsert하여 DB를 보완한다. GPT 판별 결과의 신뢰도를 확보하기 위해, 여러 출처의 텍스트에서 추출된 결과가 일치하는 경우 confidence_score를 높게 설정하고, 상충하는 경우 NEEDS_REVIEW 상태로 전환하여 관리자 검토 큐에 등록하는 불확실성 처리 로직을 적용한다.

【기술 2】 이상 감지 및 데이터 갱신 시스템 — 신뢰성의 지속적 유지

정보의 신뢰성은 초기 구축 시점뿐 아니라 시간이 지나도 유지되어야 한다. 반려동물 동반 정책은 계절·운영 방침 변경 등으로 수시로 바뀌는 동적 정보이므로, MyPetTrip은 다음 두 가지 메커니즘으로 정보 최신성을 관리한다.

1. 데이터 이상 감지 및 갱신: 사용자 신고 누적(30일 내 3건 이상), 방문·클릭 트래픽의 급격한 감소(전월 대비 60% 이상), 수집 텍스트 내 부정적 정책 관련 키워드('입장 불가', '정책 변경') 급증 등 다차원 신호를 복합 분석하여 정보 오류 가능성이 높은 장소를 NEEDS_REVIEW 상태로 전환한다. 전체 데이터를 전수 조사하는 대신 고위험군에만 관리 리소스를 집중함으로써 운영 효율성을 확보한다.

2. 반려동물 동반 정책은 계절·운영 방침 변경 등으로 수시로 바뀌는 동적 정보다. 이를 해결하기 위해 주기적인 재크롤링을 통해 기존 수집 장소에 대해 최신 블로그·후기를 재수집하고, GPT 재판별을 수행하여 DB를 갱신하는 파이프라인을 구성한다. 재판별 결과가 기존 DB와 상충하는 경우(예: 이전에는 '대형견 가능'이었으나 새 후기에서 '대형견 불가' 언급) 해당 필드를 NEEDS_REVIEW 상태로 전환하고, 변경 이력을 Firestore에 버전 관리 형태로 기록한다. 관리자는 이 큐만 집중적으로 검토하면 되므로, 전수 조사 없이 데이터 품질을 유지할 수 있다.

【기술 3】 반려동물 프로필 기반 GPT 챗봇 추천 — 도메인 특화 RAG 구조

반려동물 특성 기반 추천은 단순히 '조용한 카페를 추천해줘'가 아니라, '소음에 민감하고 노령인 소형견을 동반한 보호자에게 계단 없고 한산한 실내 카페를 추천해줘'가 실현되어야 한다. 이를 위해 MyPetTrip은 다음과 같은 구조를 채택한다.

1. 반려동물 프로필 컨텍스트 주입: 앱에 등록된 반려동물의 크기·나이·활동성·소음 민감도·사회성·건강 상태 등의 프로필 정보가 GPT 시스템 프롬프트에 삽입된다. 사용자는 매번 조건을 직접 입력하지 않아도, 챗봇이 이미 해당 반려동물의 특성을 인지한 상태에서 대화를 시작한다.
2. ~~GPT Function Calling 기반 RAG 구조~~ JSON 필터 기반 RAG 구조: 사용자가 자연어로 여행 조건을 입력하면(예: '경기도 가평에서 우리 강아지와 함께 갈만한 카페 추천해줘'), ~~GPT는 Function Calling을 통해 Firestore DB에 조건에 부합하는 장소 검색 함수를 호출하고, 반환된 검증 데이터를 근거로 추천 텍스트를 생성한다.~~ GPT는 해당 질의를 분석하여 지역 좌표, 카테고리, 반려동물 조건 등이 포함된 구조화된 JSON 필터를 생성한다. 앱은 GPT 응답에서 JSON 필터를 파싱한 뒤, 이를 Firestore 장소 데이터 조회 조건으로 사용한다. 실제 추천 장소 목록은 Firestore에 저장된 검증 데이터에서 조회되며, 조회 결과는 챗봇 화면의 추천 장소 카드로 출력된다. 이를 통해 LLM의 환각 문제를 구조적으로 차단하고, 실제 존재하며 해당 반려동물이 이용 가능한 장소만을 추천하는 신뢰성을 보장한다.

2.4 기대 효과 및 의의

2.4.1 사용자 관점의 기대 효과

(1) 방문 실패 경험 감소 및 정보 신뢰성 향상

기존 서비스에서 빈번하게 발생하는 정보 불일치 문제를 해결하기 위해, 공공데이터와 검증된 정보를 교차 확인하는 데이터 관리 체계를 구축한다. 이를 통해 반려동물 동반 가능 여부뿐만 아니라 상세 이용 정책을 구조적으로 제공함으로써 사용자의 방문 실패 경험을 방지하고 서비스의 신뢰도를 제고한다.

(2) 자연어 기반 여행 계획 수립의 획기적 간소화

기존에는 숙소·카페·식당·교통수단 정보를 각기 다른 플랫폼에서 직접 검색하고 반려견 동반 가능 여부를 일일이 확인해야 했다. MyPetTrip에서는 '경기도 가평에서 우리집 강아지와 갈만한 카페 추천해줘'와 같은 자연어 한 문장으로 이 모든 과정이 대체된다. GPT 챗봇의 대화형 인터페이스를 통해 추가 조건을 단계적으로 반영하며 일정을 조율할 수 있어, 여행 준비 과정의 부담이 크게 줄어든다.

(3) 반려동물 특성을 반영한 진정한 맞춤 추천 실현

등록된 반려동물 프로필(크기, 성향, 활동성 등)이 GPT 챗봇의 컨텍스트로 주입되어, 동일한 질문이라도 대형견 보호자와 소형견 보호자에게 전혀 다른 최적의 장소가 추천된다.

2.4.2 기술적 의의

(1) 비정형 텍스트에서 정형 데이터를 추출하는 GPT 기반 파이프라인 구현

반려동물 동반 정책은 공공 API 외에도 다양한 텍스트 형태로 존재한다. 본 프로젝트는 단순한 동반 가능 여부 확인을 넘어, 복잡한 텍스트 정보로부터 체중 제한, 출입 가능 구역 등 세부 정책을 추출하여 구조화된 데이터로 변환하는 과정을 포함한다. 이는 대규모 언어 모델(LLM)을 단순 응답 도구가 아닌 데이터 정제 및 추출 엔진으로 활용하는 실용적 컴퓨터공학적 응용 사례를 제시한다.

(2) 도메인 특화 RAG 구조를 통한 LLM 환각 문제 해결

일반 GPT 챗봇에 ‘가평 반려견 동반 카페 추천해줘’ 라고 질문하면 실제로 존재하지 않거나 잘못된 장소를 자신 있게 추천하는 환각 문제가 발생할 수 있다. ~~MyPetTrip은 GPT Function Calling을 통해 추천 생성 시 반드시 자체 DB를 조회하도록 설계한 RAG 구조를 적용함으로써, LLM 기반 서비스에서 가장 핵심적인 신뢰성 문제를 공학적으로 완화한다.~~ MyPetTrip은 GPT가 사용자의 자연어 질의를 Firestore 조회용 JSON 필터로 변환하고, 앱이 해당 필터를 기반으로 자체 DB의 장소 데이터를 조회하는 RAG 구조를 적용함으로써, LLM 기반 서비스에서 가장 핵심적인 신뢰성 문제를 공학적으로 완화한다.

~~(3) 동적 정보의 최신화 데이터 엔지니어링~~

(3) Firebase 기반 서버리스 구조를 활용한 동적 정보 관리

수시로 변경되는 실세계 정보를 효율적으로 관리하기 위해 Firebase 기반의 서버리스 아키텍처를 활용한다. ~~Cloud Functions를 활용하여 데이터 수집부터 가공, 저장까지 이어지는 프로세스를 설계함으로써 데이터의 생애주기를 체계적으로 관리하고, 시스템 운영의 효율성을 확보한다.~~ 사용자 인증은 Firebase Authentication으로 처리하고, 장소 정보, 리뷰, 신고 데이터는 Firestore Database에 저장한다. 반려동물 프로필과 관심 장소 데이터는 사용자 UID 기준의 로컬 저장소에서 관리되며, AI 추천 및 장소 탐색 시 필터

조건으로 활용된다. 이를 통해 별도의 물리 서버를 구축하지 않고도 데이터 저장, 조회, 동기화가 가능한 구조를 확보하며, 서비스 운영에 필요한 데이터 흐름을 체계적으로 관리할 수 있다.

(4) 공공데이터와 민간 비정형 데이터의 이중 데이터 통합

출처와 형식이 상이한 공공데이터 API의 정형 데이터와 민간의 비정형 정보를 하나의 표준화된 Firestore 스키마로 통합하는 과정을 수행한다. 이는 현실적인 서비스 개발 환경에서 직면하는 데이터 퓨전(Data Fusion) 문제에 대한 실용적인 기술적 해법을 제시한다.

2.5 주요 기능 리스트

MyPetTrip은 반려동물 여행 준비 과정에서 발생하는 정보 탐색의 불편함과 계획 수립의 비효율성을 해결하기 위해 설계되었다. 본 서비스의 주요 기능은 단순한 장소 나열을 넘어, 반려동물의 세부 조건과 장소의 제한 사항을 구조화하여 매칭하고, 이를 AI 챗봇 상담으로 연결하는 유기적 체계로 구성된다.

2.5.1 주요 기능 요약 및 구현 방식

NO	주요 기능	상세 내용	구현 방식
1	반려동물 프로필 등록	크기, 견종, 활동성 등 개별 특성 데이터화	사용자 UID 기준의 로컬 저장소 저장 및 프로필 관리
2	지도 기반 장소 탐색	현재 위치 중심의 반려동물 친화 장소 시각화 및 조건 필터링	Google Maps API 및 Geocoding API를 통한 정밀 좌표 매칭
3	구조화된 장소 정보 제공	체중 제한, 입출입 규정 등 세부 정책 정보의 명확한 표시	TourAPI 데이터와 비정형 정보를 통합한 정형 DB 구축
4	AI 챗봇 여행 상담	자연어 질의를 통한 맞춤형 장소 추천 및 여행 코스 제안	OpenAI GPT API 기반 JSON 필터 생성 및 Firestore 추천 조회
5	선호 장소 저장 및 관리	마음에 드는 장소 북마크 및 개인 여행 리스트 관리	사용자 UID 기준의 로컬 저장소 저장 및 Firestore 장소 데이터 매칭 조회

2.5.2 기능별 세부 설계 내용

(1) 맞춤형 반려동물 프로필 등록

본 기능은 사용자가 자신의 반려동물에 대한 기본 정보와 성향 데이터를 직접 입력하고 관리할 수 있도록 설계된다. 여기에는 반려견의 활동성(activityLevel), 크기(petSize) 등의 요소가 포함되며, 해당 데이터는 사용자 UID 기준의 로컬 저장소에 저장되어 이후 모든 장소 추천 및 챗봇 상담의 핵심 컨텍스트로 활용된다. 예를 들어, 소형견이며 활동성이 낮은 반려동물과 대형견이며 활동성이 높은 반려동물은 각각의 특성에 최적화된 서로 다른 탐색 결과를 제공받게 된다.

(2) 위치 기반 지도 탐색 및 시각화

사용자는 단순 목록 형태보다 실제 위치와 주변 분포를 함께 확인하기를 원하므로, MyPetTrip은 현재 위치를 중심으로 주변의 반려동물 동반 가능 장소를 지도 위에 시각적으로 표시한다. 특히 Geocoding API를 활용하여 수집된 장소의 텍스트 주소를 정밀한 위경도 좌표로 변환함으로써, 지도상에서 오차 없는 마커 배치를 구현한다. 이를 통해 사용자는 숙소, 카페, 식당 등 주요 시설의 밀집도를 한눈에 파악하고 실제 이동 효율성을 고려한 여행 계획을 수립할 수 있다. 처음 방문하는 지역에서도 반려동물 동반 정책이 검증된 장소들을 지도 기반으로 즉시 확인할 수 있어 탐색의 부담을 획기적으로 줄여준다.

(3) AI 챗봇 기반 대화형 여행 상담

본 기능은 사용자가 복잡한 검색 조건을 입력하는 대신, 자연어 형태의 질문을 통해 필요한 정보를 얻을 수 있도록 지원한다. 사용자가 “가평에서 우리 강아지와 갈 만한 카페 추천해줘”와 같이 질의하면, 시스템은 등록된 반려동물 프로필과 Firestore DB 내의 검증된 장소 정보를 결합하여 맞춤형 결과를 도출한다. 이는 단순 키워드 검색의 한계를 넘어 사용자의 상황과 맥락을 반영한 정교한 탐색을 가능하게 하며, 여행 준비 시간을 단축시키는 핵심 역할을 한다.

(4) 선호 장소 등록 및 개인화 리스트 관리

사용자는 탐색 과정에서 발견한 유용한 장소들을 개별 북마크 기능을 통해 저장하고 언제든지 재확인할 수 있다. 저장된 장소 데이터는 사용자의 계정 정보와 연동되어 단순한 리스트 보관을 넘어, 향후 실제 여행 동선을 짜거나 챗봇 상담 시 우선 순위 데이터로 활용될 수 있도록 설계된다. 이 기능은 여러 플랫폼을 오가며 정보를 다시 찾아야 했던 기존의 번거로움을 해결하고 서비스 내부에서 통합적인 여행 관리를 가능하게 한다.

종합하면, MyPetTrip의 주요 기능들은 독립적으로 존재하는 것이 아니라 구조화된 데이터와 사용자 프로필을 기반으로 상호 연결되어 작동한다. 결과적으로 본 서비스는 단순한 정보 제공 앱을 넘어, 반려동물의 특성과 보호자의 상황을 공학적으로 분석하여 가장 정확하고 편리한 여행 준비를 지원하는 맞춤형 추천 서비스로 기능하도록 설계되었다.

3. Project Design

3.1 요구사항 정의

본 절에서는 앞서 도출한 주요 기능을 기반으로, MyPetTrip 서비스가 실제 구현 과정에서 충족해야 할 기능적 요구사항을 구체적으로 정의한다. 본 서비스는 반려동물 동반 여행 과정에서 발생하는 정보 탐색의 비효율성과 이용 조건 확인의 어려움을 해결하는 것을 목표로 하며, 이를 위해 요구사항은 크게 반려동물 프로필 관리, 지도 기반 장소 탐색, AI 챗봇 상담, 사용자 편의 기능으로 구분하였으며, 각 기능은 상호 연계되어 작동한다.

3.1.1 기능적 요구사항 명세

기능적 요구사항 명세는 다음과 같다.

분류	ID	요구사항 명칭	상세 구현 내용
프로필 관리	F-01	반려동물 데이터 등록 및 관리	이름, 나이, 성별, 체중 등 기본 정보와 활동성 등 성향 정보를 로컬 저장소에 저장하며, 수정 및 삭제 시 즉시 반영함.
	F-02	조건 기반 입장 판별	등록된 체중 및 견종 정보를 바탕으로 장소별 입장 가능 여부를 시스템이 자동 필터링하여 판단함.
지도 탐색	F-03	위치 기반 장소 시각화	Google Maps API를 통해 사용자의 현 위치 주변 동반 가능 장소를 지도 위에 핀(Pin) 형태로 표시함.
	F-04	상세 정보 및 인터페이스	지도와 리스트 뷰를 선택적으로 제공하며, 마커 선택 시 거리, 위치를 포함한 장소 상세 정보를 노출함.
AI 챗봇	F-05	자연어 기반 의도 분석	OpenAI GPT API를 통해 질의에서 지역 정보, 반려동물 조건, 여행 목적 및 선호 키워드를 추출함.
	F-06	RAG 기반 추천 생성	추출된 정보와 DB 내 검증 데이터를 결합하여 사용자 조건에 부합하는 최적의 장소를 대화형으로 제시함.

편의 기능	F-07	관심 장소 저장 및 관리	직관적인 UI를 통해 관심 장소를 저장하고, 별도의 목록에서 위치 및 세부 이용 제한 사항을 상시 조회함.
-------	------	---------------	---

3.1.2 사용자 시나리오 및 운영 요구사항

본 절에서는 정의된 기능적 요구사항이 실제 서비스 환경에서 사용자에게 어떠한 가치를 제공해야 하는지 운영 측면의 지침을 기술한다.

(1) 데이터 기반 개인화

사용자는 다수의 반려동물 프로필을 생성하고 관리할 수 있어야 하며, 시스템은 저장된 체중, 성향, 활동성 등의 데이터를 모든 탐색 및 추천 로직의 핵심 입력값으로 활용해야 한다. 이는 사용자별로 최적화된 맞춤형 결과를 생성하는 기반이 된다.

(2) 공간적 의사결정 지원

지도 기반 탐색 시 사용자의 현재 위치를 실시간으로 반영하여 주변 장소 정보를 시각화해야 한다. 특히 등록된 프로필 조건에 부합하는 장소를 우선적으로 노출함으로써, 사용자가 이동 동선을 짜는 과정에서 겪는 탐색 부담을 최소화하고 효율적인 의사결정을 지원해야 한다.

(3) 신뢰성 있는 정보 제공

장소의 상세 페이지에는 단순한 위치 정보뿐만 아니라 반려동물 동반 시 지켜야 할 체중 제한, 출입 가능 구역, 이용 정책 등 방문 실패를 방지하기 위한 핵심 데이터가 반드시 포함되어야 한다. 이는 공공데이터와 검증된 정보를 통합하여 정보의 신뢰성을 확보하는 것을 원칙으로 한다.

(4) 대화형 인터페이스 확장

사용자가 복잡한 검색 필터를 설정하는 대신 자연어 질의를 통해 원하는 여행 조건을 입력할 수 있도록 지원한다. 시스템은 이를 분석하여 내부 DB의 검증된 데이터와 결합함으로써, 생성형 AI의 고질적인 문제인 환각 현상 없이 정확한 정보를 바탕으로 최적의 여행 코스를 제안해야 한다.

3.1.3 Feature별 작동 시나리오 및 예상 결과

주요 기능(Feature)들이 설계 의도대로 작동하는지 검증하기 위한 구체적인 실험 시나리오와 예상 결과는 다음과 같다.

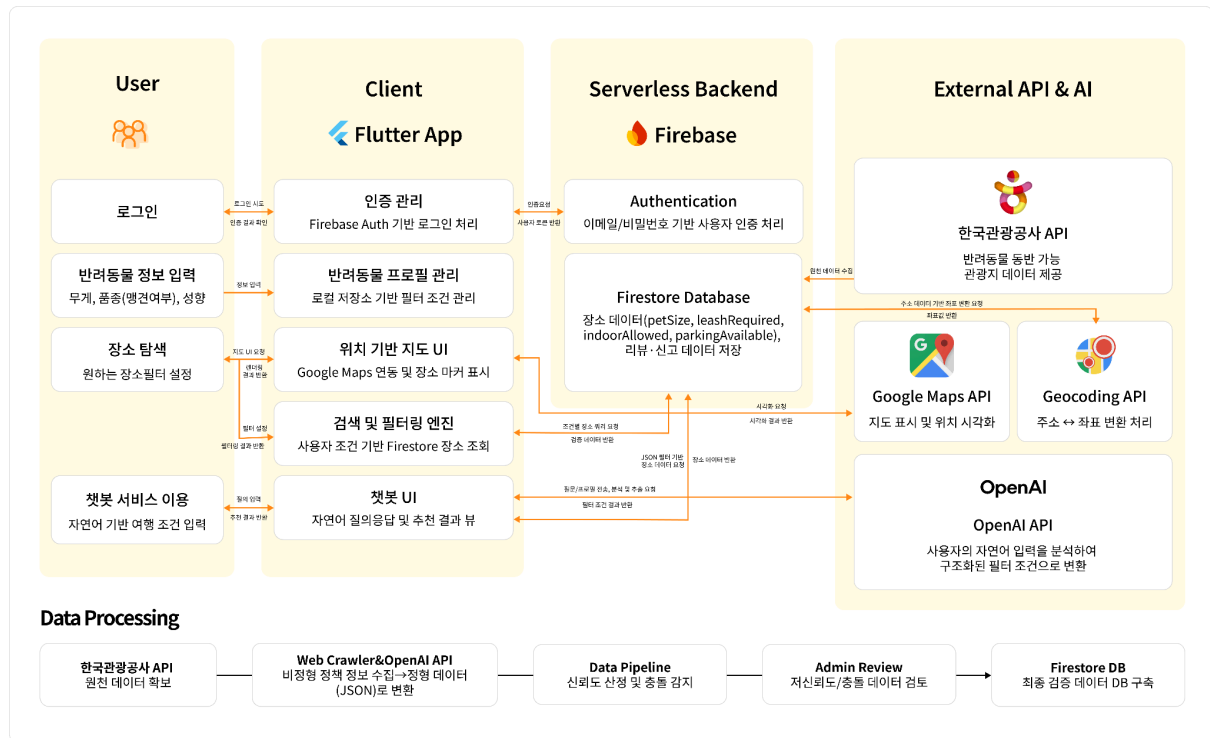
분류	ID	실험 시나리오	예상 결과
프로필 관리	F-01	사용자가 이름 'OO이', 무게 '7kg', 성향 '사회성 높음'으로 프로필을 등록함	로컬 저장소에 해당 속성값이 저장되며, 이후 모든 검색 쿼리의 기본 필터값으로 고정됨
	F-02	소형견 출입이 제한된 대형견 전용 운동장 장소를 검색함	OO이의 프로필(7kg, 소형견)과 장소 제한 데이터가 매칭되어, 이용 불가능한 장소로 자동 판별됨
지도 탐색	F-03	앱 실행 후 '내 주변 장소 보기' 버튼을 클릭함	Google Maps API를 통해 현재 위경도 좌표가 인식되고, 반경 5km 이내의 동반 가능 장소 핀이 출력됨
	F-04	지도상의 특정 숙소 마커를 터치함	하단 시트에 해당 숙소의 상세 운영 정책(예: '목줄 필수', '실외 이용 가능') 정보가 Firestore로부터 호출되어 표시됨
AI 챗봇	F-05	자연어로 "OO이랑 갈 수 있는 가평 숙소 알려줘"라고 질의함	OpenAI API가 문장을 분석하여 '장소: 가평', '카테고리: 숙박' 키워드를 추출함. 시스템은 'OO이'라는 이름을 통해 현재 활성화된 반려동물 프로필(예: 7kg, 소형견, 사회성 높음)을 자동으로 컨텍스트에 주입함
	F-06	위 질의에 대해 최적의 장소 추천을 요청함	RAG 구조를 통해 DB내 실존하는 '소형견 가능' 카페를 필터링하고, GPT는 "OO이는 소형견이라 실내 이용이 가능합니다"와 같은 환각 없는 맞춤형 추천을 생성함 RAG 구조를 통해 DB내 실존하는 '소형견 가능' 숙소를 필터링하고, GPT는 "OO이는 소형견이므로 해당 숙소의 실내 객실 이용 조건에 부합합니다"와 같은 환각 없는 맞춤형 추천을 생성함

편의 기능	F-07	검색 결과 중 특정 숙소의 '저장' 아이콘을 클릭함	해당 장소의 ID가 사용자 UID 기준의 로컬 저장소에 저장되어 마이페이지에서 확인 가능해짐
-------	------	------------------------------	---

3.2 전체 시스템 구성

본 절에서는 반려동물 동반 여행 서비스 My Pet Trip의 기술적 구현을 위한 전체 시스템 구조를 기술한다. 시스템은 별도의 물리 서버를 구축하지 않고 클라우드 자원을 효율적으로 활용하는 Firebase 기반의 Serverless Backend 아키텍처를 채택하였다. 전체 구조는 User, Client(Flutter App), Serverless Backend(Firebase), 그리고 External API & AI 계층의 유기적 결합으로 설계되었다.

<시스템 구성도 - 하단 Data Processing 파트 수정>



3.2.1 계층별 주요 모듈 및 역할

전체 시스템은 역할에 따라 다음과 같이 4개의 계층으로 구분된다.

(1) 사용자 계층 (User)

사용자는 서비스의 최종 이용 주체로서 시스템과 상호작용하며 개인화된 데이터를 생성하는 역할을 수행한다.

- ① **인증 및 접근:** ~~구글 계정 기반의 소셜 로그인을 통해 시스템에 접속하며, 보안이 유지된 개인 세션을 생성한다.~~ 사용자는 **Firebase Authentication**을 통해 이메일/비밀번호 기반으로 로그인하며, 인증 완료 후 발급되는 고유 **UID**를 기준으로 개인 세션과 사용자 데이터를 구분한다.

- ② **반려동물 프로필 구축:** 개별 반려동물의 무게, 품종(맹견 여부 포함), 활동성 등 성향 정보를 상세히 등록하여 맞춤형 서비스 제공을 위한 기초 컨텍스트를 구축한다.
- ③ **대화형 서비스 이용:** 자연어 질의를 통해 반려동물 동반 여행지에 대한 정보를 탐색하고, 실시간으로 생성되는 최적의 여행 코스를 제안받는다.

(2) 프론트엔드 앱 계층 (Client)

크로스 플랫폼 프레임워크인 **Flutter**를 기반으로 하며, 사용자 경험(UX)을 극대화하기 위한 다양한 기능 모듈로 구성된다.

- ① **인증 및 프로필 관리 모듈:** Firebase Auth와 연동하여 사용자의 로그인 상태를 유지하고, 입력된 반려동물 데이터는 PetProfileService를 통해 사용자 UID 기준의 로컬 저장소에서 관리한다.
- ② **위치 기반 지도 시각화 모듈:** Google Maps API를 활용하여 현재 위치 중심의 장소 탐색 인터페이스를 제공하며, 조건에 부합하는 장소를 마커(Marker) 형태로 표시한다.
- ③ **지능형 검색 및 챗봇 엔진:** OpenAI API와 연동된 대화형 UI를 통해 사용자의 의도를 분석하고, 추출된 필터 조건에 따른 검색 결과를 사용자에게 동적으로 렌더링한다.

(3) 백엔드 서비스 계층 (Firebase)

시스템의 핵심 로직과 데이터 저장을 담당하는 서버리스 백엔드이다. **사용자의 인증 정보와 반려동물 프로필, 그리고 정제된 장소 데이터를 실시간으로 관리하고 저장하는 각 모듈의 세부 역할은 다음과 같다.**

모듈 명칭	주요 기능 및 역할	비고
Authentication	이메일/비밀번호 기반 사용자 인증 처리를 수행하며, 사용자별 고유 식별자(UID)를 관리함	사용자 보안 및 세션 관리
Firestore Database	정제된 장소 데이터, 리뷰 및 신고 데이터를 관리하는 NoSQL 데이터베이스이며, AI 추천 및 지도 탐색 시 장소 데이터 조회에 활용됨	실시간 데이터 동기화 지원

Firebase 계층은 사용자 인증과 장소 데이터 관리를 담당하는 중심 백엔드 역할을 수행한다. Authentication에서 발급된 UID는 앱 내 로컬 저장소에서 사용자별 반려동물 프로필과 관심 장소 데이터를 구분하는 기준으로 활용되며, Firestore는 정제된 장소 데이터, 리뷰, 신고 데이터 등을 저장하고 AI 추천 및 지도 탐색 기능에 필요한 장소 조회를 지원한다. 이를 통해 Client 계층에서 발생한 사용자 입력과 외부 API 기반 장소 데이터가 서비스 기능에 맞게 분리 관리된다.

(4) 외부 서비스 및 AI 계층 (External API)

시스템의 지능형 서비스와 기초 데이터 공급을 담당한다. 시스템의 지능형 기능을 지원하는 각 모듈의 상세 역할은 다음과 같다.

모듈 명칭	주요 기능 및 역할	비고
OpenAI API	사용자의 자연어 입력을 분석하여 의도를 파악하고, 이를 시스템이 이해할 수 있는 구조화된 필터 조건으로 변환하는 NLP 엔진 역할을 수행함	RAG 기반 맞춤형 추천의 핵심
Google Maps & Geocoding API	Google Maps 연동을 통한 지도 인터페이스 시각화 및 장소 주소와 위경도 좌표 간의 변환 처리 담당	위치 기반 서비스 및 데이터 정교화
한국관광공사 API	반려동물 동반이 가능한 전국 관광지 및 시설의 원천 데이터 (장소명, 주소 등)를 시스템에 공급	고신뢰도 데이터베이스 구축의 기초

External API & AI 계층은 시스템 내부 데이터만으로 처리하기 어려운 자연어 분석, 위치 변환, 지도 시각화, 원천 장소 데이터 수집을 담당한다. OpenAI API는 사용자의 자연어 질의를 구조화된 필터 조건으로 변환하고, Google Maps 및 Geocoding API는 장소 주소와 좌표를 연결하여 지도 기반 탐색을 가능하게 한다. 한국관광공사 API는 반려동물 동반 장소의 기초 데이터를 제공하며, 이 데이터는 Firestore에 저장된 후 AI 추천과 지도 탐색 기능에서 공통으로 활용된다.

3.2.2 시스템 데이터 흐름 및 상호작용

시스템 내부의 데이터 흐름은 사용자의 요청에 따라 크게 다음의 **세 가지 4가지** 경로로 진행된다.

(1) 사용자 인증 및 **프로필 흐름 보안 세션 흐름**

사용자의 로그인 시도는 Client의 인증 관리 모듈을 거쳐 Firebase Authentication과 양방향으로 통신하며, **등록된 반려동물 정보는 데이터 저장 흐름을 통해 Firestore에 기록된다.** 이를 통해 고유 UID를 발급받고 안전한 세션을 생성한다. 인증 결과와 사용자 정보가 반환되면 사용자는 서비스 메인 화면에 접근할 수 있는 권한을 획득한다.

(2) 반려동물 프로필 등록 및 개인화 흐름

사용자가 입력한 반려동물의 무게, 품종, 성향 등 세부 데이터는 Flutter를 통해 구조화되어 사용자 UID 기준의 로컬 저장소에 기록된다. 등록된 프로필 정보는 데이터 저장 흐름을 거쳐 시스템 내에 유지되며, 이후 모든 탐색 요청 시 검색 쿼리에 동적으로 반영되어 사용자별 맞춤형 필터링 환경을 조성하는 기초가 된다.

(3) AI 기반 추천 및 챗봇 흐름

챗봇 UI를 통해 입력된 사용자의 질의는 OpenAI API로 전달되어 필터 조건으로 변환된다. ~~추출된 조건은 Firestore의 검증 데이터와 매칭되어 다시 사용자에게 맞춤형 추천 결과로 반환되는 RAG(Retrieval Augmented Generation) 구조를 형성한다.~~ 시스템은 해당 조건을 바탕으로 Firestore의 검증된 데이터와 매칭하여 장소 정보를 조회하며, 시스템은 해당 조건을 바탕으로 Firestore의 검증된 장소 데이터를 조회하며, 조회된 결과는 챗봇 화면의 추천 장소 카드로 출력된다. 이러한 검색 증강 생성(RAG) 구조를 통해 LLM의 환각 현상을 구조적으로 차단하고, 사용자 특성이 반영된 신뢰성 높은 맞춤형 추천 결과를 반환한다.

(4) 위치 기반 탐색 흐름

~~장소 탐색 요청은 검색 및 필터링 엔진을 통해 Firestore에 쿼리되며, 반환된 결과는 Google Maps API를 통해 지도상의 마커로 시각화되어 사용자에게 제공된다.~~

장소 탐색 요청 시 Flutter는 실시간 GPS 위경도 좌표를 인식하여 Firestore DB에서 조건에 부합하는 장소 데이터를 추출한다. 이때 추출된 장소의 주소 정보는 Geocoding API를 통해 정밀한 위경도 좌표로 변환되는 과정을 거친다. 최종적으로 변환된 좌표값은 Google Maps API로 전달되어 지도 UI 상의 마커로 시각화되어 사용자에게 제공된다.

이와 같이 각 계층은 독립적으로 동작하는 것이 아니라, 사용자 입력이 Client를 통해 Firebase에 저장되고, 필요한 경우 External API 및 AI 계층과 연동되어 다시 사용자에게 추천 결과로 반환되는 순환 구조를 이룬다. 반려동물 프로필 데이터는 사용자 UID 기준의 로컬 저장소에서 관리되고, 장소 정책 데이터는 Firestore를 중심으로 관리되며, 두 데이터는 AI 챗봇 추천과 지도 탐색 기능에서 함께 활용된다.

3.2.3 데이터 신뢰성 확보를 위한 처리 파이프라인

정보의 최신성과 정확성을 유지하기 위해 백그라운드에서 데이터 처리 파이프라인이 구동된다.

(1) 수집 및 변환

~~한국관광공사 API로부터 원천 데이터를 수집하고, Geocoding API를 통해 주소를 정밀 좌표 데이터로 변환한다.~~

(2) 구조화 및 DB 구축

~~수집된 비정형 정보를 반려동물 특성에 따른 서비스 전용 필터값으로 구조화하여, 최종적으로 검증된 장소 데이터를 Firestore DB에 구축한다.~~

(1) 다중 출처 데이터 수집 및 변환

한국관광공사 TourAPI 4.0으로부터 정형 원천 데이터를 수집하고, 웹 크롤러를 통해 민간(공식 웹, SNS) 비정형 데이터를 확보한다.

(2) LLM 기반 구조화 및 신뢰도 산정

GPT-4o-mini를 활용하여 비정형 정책 텍스트를 9개 핵심 필드(반려견 크기, 출입 공간 등)로 구조화한다. 추출된 데이터는 Confidence Score 로직에 따라 최신성과 출처 신뢰도를 바탕으로 수치화된다.

(3) 충돌 감지 및 검증 DB 구축

다중 출처 간 정보 상충이 발생할 경우 충돌 감지 알고리즘을 통해 NEEDS_REVIEW 상태로 분류하여 관리자 검토를 거친다. 최종 검증이 완료된 고신뢰도 데이터만을 추출하여 Firestore DB에 구축한다.

3.3 주요 엔진 및 기능 설계

본 절에서는 MyPetTrip의 핵심 기능을 수행하는 주요 SW 모듈과 각 모듈의 내부 설계 구조를 기술한다. MyPetTrip은 단순히 반려동물 동반 가능 장소 목록을 제공하는 서비스가 아니라, 사용자 반려동물 프로필, 위치 정보, 장소 데이터, OpenAI API 기반 자연어 처리 결과를 결합하여 맞춤형 탐색과 추천을 제공하는 시스템이다. 따라서 본 서비스의 주요 기능은 하나의 모듈만으로 구현되지 않고, 사용자 입력, Firebase 기반 인증 및 데이터 저장, 외부 API 연동, AI 기반 필터 JSON 생성, Firestore 추천 조회, 추천 결과 시각화 과정이 순차적으로 연결되어 동작하도록 설계하였다.

NO	모듈명	관련 기능	주요 역할
1	반려동물 프로필 관리 및 필터링 모듈	F-01, F-02, F-05, F-06	사용자의 반려동물 정보를 저장하고, AI 추천 및 장소 탐색 시 필터 조건으로 활용
2	위치 기반 지도 탐색 및 시각화 모듈	F-03, F-04	현재 위치와 장소 좌표를 기반으로 주변 장소를 지도에 표시
3	검색 및 개인화 필터링 엔진	F-02, F-03, F-06	사용자 조건, 반려동물 프로필, 장소 속성을 결합하여 적합한 장소 후보 선별
4	AI 필터 JSON 생성 엔진	F-05, F-06	자연어 질의를 분석하여 지역 좌표와 반려동물 조건이 포함된 JSON 필터 생성
5	Firestore 추천 조회 모듈	F-06	AI가 생성한 JSON 필터를 기반으로 Firestore에서 추천 장소 조회
6	추천 결과 시각화 모듈	F-06	AI 답변과 추천 장소 카드를 챗봇 화면에 출력
7	관심 장소 저장 모듈	F-07	사용자가 선택한 장소 ID를 사용자 UID 기준의 로컬 저장소에 저장 및 관리
8	데이터 신뢰성 관리 모듈	데이터 관리	장소 데이터의 최신성, 검토 필요 상태, 외부 API 기반 데이터 보완 관리

위 표의 각 모듈은 독립적으로 존재하는 것이 아니라, 사용자의 요청 흐름에 따라 서로 연결된다. 예를 들어 사용자가 AI 챗봇에 “OO이랑 가평 여행 갈 건데 장소 추천해줘”라고 입력하면, AI 필터 JSON 생성 엔진은 자연어 질의에서 지역과 반려동물 조건을 추출하고, Firestore 추천 조회 모듈은 해당 JSON 필터를 기반으로 장소 데이터를 조회한다. 이후 추천 결과 시각화 모듈은 조회된 장소 목록을 챗봇 화면의 추천 장소 카드 형태로 출력한다.

3.3.1 반려동물 프로필 관리 및 필터링 모듈

반려동물 프로필 관리 및 필터링 모듈은 사용자가 등록한 반려동물 정보를 저장하고, 이후 장소 탐색 및 AI 챗봇 추천 과정에서 맞춤형 조건으로 활용하기 위한 모듈이다. 사용자는 반려동물의 이름, 나이, 성별, 중성화 여부, 품종, 맹견 여부, 체중, 크기, 활동성, 여행 체크리스트 등의 정보를 등록할 수 있다.

AI 채팅 기능에서는 PetProfileService를 통해 저장된 반려동물 프로필 목록과 현재 선택된 프로필을 로드한 뒤, 해당 데이터를 OpenAiService의 입력값으로 전달한다. OpenAiService는 사용자 질의, 대화 이력, 반려동물 프로필 정보를 함께 사용하여 Firestore 추천 조회에 필요한 필터 JSON을 생성한다. 생성된 JSON에는 위치 좌표(mapX, mapY)뿐만 아니라 petName, petWeight, petSize, isFierceDog, activityLevel, indoorAllowed, parkingAvailable 등 추천 조건으로 활용될 수 있는 프로필 기반 필드가 포함된다. 이를 통해 사용자의 자연어 질의가 반려동물 특성과 결합된 구조화된 검색 조건으로 변환되며, 이후 Firestore 추천 조회 과정에서 맞춤형 장소 후보를 선별하는 데 활용된다.

필드명	설명	활용 위치
petName	반려동물 이름	질문 내 반려동물 이름 매칭 및 챗봇 추천
petAge	반려동물 나이	노령견 여부 등 추천 조건 판단
petGender	반려동물 성별	프로필 정보 관리
isNeutered	중성화 여부	프로필 정보 관리
petBreed	품종 정보	견종 또는 맹견 여부 판단 보조
isFierceDog	맹견 여부	입장 제한 조건 필터링
petWeight	체중	체중 제한 장소 필터링
petSize	소형견, 중형견, 대형견 구분	장소 이용 가능 여부 판단
activityLevel	활동성 수준	산책, 운동장, 조용한 장소 등 추천 기준
travelChecklist	여행 시 필요한 조건 목록	사용자의 추가 선호 조건 반영
isOffLeash	오프리쉬 가능 여부	반려견 운동장, 야외 공간 추천
indoorAllowed	실내 이용 가능 여부	실내 동반 가능 장소 필터링
parkingAvailable	주차 가능 여부	편의 조건 기반 장소 필터링

본 모듈의 핵심은 프로필 데이터가 단순 저장용 데이터가 아니라, AI 추천과 장소 검색 과정의 필터 조건으로 활용된다는 점이다. 예를 들어 사용자가 “OO이랑 갈 수 있는 가평 숙소 알려줘”라고 입력하면, 시스템은 사용자의 최근 메시지에 ‘OO이’라는 이름이 포함되어 있는지 확인하고, 저장된 반려동물 프로필 중 이름이 일치하는 프로필을 찾는다. 일치하는 프로필이 존재하면 해당 프로필의 체중, 크기, 활동성, 맹견 여부 등의 값이 AI 필터 JSON에 병합된다.

이러한 구조를 통해 사용자는 매번 체중, 크기, 성향 조건을 직접 입력하지 않아도 된다. 또한 동일한 지역과 카테고리를 검색하더라도 반려동물의 크기, 체중, 맹견 여부, 실내 이용 가능 조건에 따라 서로 다른 추천 결과를 받을 수 있다.

3.3.2 AI 필터 JSON 생성 엔진

AI 필터 JSON 생성 엔진은 사용자의 자연어 질의를 분석하여 Firestore 추천 조회에 사용할 수 있는 구조화된 JSON 필터를 생성하는 모듈이다. 현재 구현에서는 OpenAI API의 Chat Completions 요청을 사용하며, 시스템 프롬프트를 통해 응답 형식을 “짧은 자연어 답변 + JSON 객체 1개”로 고정한다.

현재 구현된 필터 JSON은 장소의 세부 정책 데이터를 저장하기 위한 JSON이 아니라, 사용자의 자연어 질의를 Firestore 추천 조회 조건으로 변환하기 위한 검색 필터 JSON이다. 예를 들어 사용자가 “OO이랑 가평 여행 갈 건데 장소 추천해줘”라고 입력하면, AI는 가평 지역의 중심 좌표를 mapX, mapY에 넣고, 질문에서 확인된 반려동물 조건이나 사용자 조건을 나머지 필드에 반영한다. 조건이 명시되지 않은 항목은 null 또는 [] 상태로 유지되며, 이는 해당 조건에 제한을 두지 않는다는 의미로 사용된다.

현재 구현에서 AI 응답 JSON은 다음과 같은 형태로 생성된다.

```
{
  "mapX": 127.4883,
  "mapY": 37.8328,
  "petName": "스타트",
  "petAge": 5,
  "petGender": "F",
  "isNeutered": true,
  "petBreed": "말티즈",
  "isFierceDog": false,
  "petWeight": 5.0,
  "petSize": "S",
  "activityLevel": "L",
  "travelChecklist": ["물 자주 마시기"],
  "isOffLeash": false,
  "indoorAllowed": true,
  "parkingAvailable": true
}
```

이 구조의 핵심은 AI가 자연어 질의를 단순히 답변 문장으로만 처리하지 않고, 내부 추천 조회에 사용할 수 있는 필터 조건으로 변환한다는 점이다. 이를 통해 MyPetTrip은 대화형 인터페이스와 Firestore 기반 장소 추천 기능을 연결할 수 있다.

3.3.3 지역명 인식 및 좌표 보정 모듈

지역명 인식 및 좌표 보정 모듈은 사용자의 자연어 질의에서 지역명을 추출하고, 이를 실제 지도 좌표로 변환하기 위한 모듈이다. AI 추천 기능에서 위치 기반 장소 검색을 수행하려면 mapX, mapY 좌표값이 반드시 필요하다. 따라서 본 프로젝트는 OpenAI 응답 JSON에서 좌표값을 생성하도록 할 뿐 아니라, 사용자의 메시지에서 지역명이 감지되는 경우 Google Geocoding API를 통해 좌표를 보정하는 구조를 함께 사용한다.

현재 구현에서는 서울, 부산, 대구, 인천, 광주, 대전, 울산, 세종, 경기, 강원, 충북, 충남, 전북, 전남, 경북, 경남, 제주 등 주요 행정구역 별칭을 정규화한다. 예를 들어 사용자가 “서울 근처”, “경기 인근”, “제주도”처럼 입력하더라도 이를 각각 “서울특별시”, “경기도”, “제주특별자치도”로 변환할 수 있도록 별칭 매핑을 구성하였다.

또한 사용자의 질의에서 행정구역 표현을 찾기 위해 정규표현식을 사용한다. 지역명이 추출되면 Google Geocoding API를 통해 해당 지역의 위도와 경도를 가져오고, 그 결과를 AI 응답 JSON의 `mapX`, `mapY` 값에 반영한다.

처리 흐름은 다음과 같다.

1. 사용자가 자연어로 장소 추천을 요청한다.
2. 시스템은 최근 사용자 메시지에서 지역 표현을 탐색한다.
3. 지역 별칭이 있는 경우 표준 행정구역명으로 정규화한다.
4. 지역명이 확인되면 Google Geocoding API를 통해 좌표를 조회한다.
5. 조회된 경도와 위도를 각각 `mapX`, `mapY`에 반영한다.
6. 좌표가 포함된 필터 JSON을 Firestore 추천 조회에 사용한다.

이 모듈은 사용자가 정확한 행정구역명을 입력하지 않아도 위치 기반 추천이 가능하도록 돕는다. 또한 `mapX`, `mapY` 좌표가 존재할 때만 Firestore 추천 조회가 실행되도록 하여, 위치 정보가 불완전한 상태에서 잘못된 추천이 수행되는 것을 방지한다.

3.3.4 검색 및 맞춤형 필터링 엔진

검색 및 맞춤형 필터링 엔진은 AI 필터 JSON과 Firestore 장소 데이터를 연결하는 모듈이다. MyPetTrip의 추천 기능은 단순히 지역만 기준으로 장소를 검색하는 것이 아니라, 반려동물 프로필과 사용자가 직접 말한 조건을 함께 반영해야 한다. 따라서 AI 응답 JSON에는 위치 좌표뿐 아니라 맹견 여부, 체중, 크기, 실내 이용 가능 여부, 주차 가능 여부 등 장소 필터링에 필요한 조건이 함께 포함된다.

현재 구현에서는 OpenAI 응답 후처리 과정에서 질문에 포함된 조건을 추가로 반영한다. 예를 들어 사용자가 “맹견” 또는 “법적맹견”이라는 단어를 입력하면 `isFierceDog` 값을 `true`로 설정하고, “맹견 아님”, “맹견 아닌”, “맹견 제외”와 같은 표현이 포함되면 `isFierceDog` 값을 `false`로 설정한다. 이를 통해 AI가 놓칠 수 있는 핵심 조건을 코드 레벨에서 보정한다.

사용자 표현	반영되는 JSON 값
맹견, 법적맹견	isFierceDog = true
맹견 아님, 맹견 아닌, 맹견 제외	isFierceDog = false
실내 가능	indoorAllowed = true
주차 가능	parkingAvailable = true
오프리쉬 가능	isOffLeash = true
특정 반려동물 이름 포함	해당 반려동물 프로필 값 병합

검색 및 개인화 필터링 엔진은 이러한 JSON 값을 Firestore 추천 조회 조건으로 사용한다. 즉, 사용자의 자연어 질문은 AI 필터 JSON으로 변환되고, 이 JSON은 FirestoreService의 추천 조회 함수에 전달되어 조건에 맞는 장소 후보를 찾는 데 사용된다. 조건이 **null**인 필드는 제한 조건이 없는 것으로 해석되며, 값이 존재하는 필드만 검색 조건으로 반영된다.

3.3.5 Firestore 추천 조회 모듈

Firestore 추천 조회 모듈은 AI가 생성한 JSON 필터를 기반으로 실제 장소 데이터를 조회하는 모듈이다. AI 챗봇이 생성한 자연어 답변만으로는 실제 장소 추천의 신뢰성을 보장하기 어렵기 때문에, MyPetTrip은 AI 응답에서 JSON 필터를 추출한 뒤 Firestore의 장소 데이터 조회와 연결한다.

AI 채팅 화면에서는 OpenAI 응답 문자열에서 {와 } 사이에 있는 JSON 구간을 찾고, 이를 `jsonDecode()`를 통해 Map 형태로 변환한다. 이후 `mapX`, `mapY` 값이 정상적으로 존재하는지 확인한다. 좌표값이 없거나 파싱에 실패하면 추천 장소 조회를 수행하지 않고 빈 결과를 반환한다. 좌표값이 존재하는 경우에만 `FirestoreService.recommendTourPlacesForAi(filters)`를 호출하여 추천 장소 목록을 조회한다.

처리 흐름은 다음과 같다.

1. OpenAI API로부터 자연어 답변과 JSON 필터가 포함된 응답을 받는다.
2. 응답 문자열에서 JSON 객체의 시작과 끝 위치를 찾는다.
3. JSON 문자열을 파싱하여 필터 Map을 생성한다.
4. `mapX`, `mapY` 좌표가 존재하는지 확인한다.
5. 좌표가 유효하면 Firestore 추천 조회 함수를 호출한다.
6. 추천 장소 목록을 반환받는다.
7. 각 추천 장소에 리뷰 수 정보를 결합한다.
8. 추천 결과를 챗봇 화면의 추천 장소 카드로 출력한다.

이 구조는 AI가 장소를 임의로 생성하지 않고, Firestore에 저장된 장소 데이터를 기반으로 추천 결과를 구성하도록 만드는 역할을 한다. 즉, GPT는 사용자의 질의를 검색 가능한 JSON 필터로 변환하고, 실제 추천 장소 목록은 Firestore 데이터 조회 결과를 통해 생성된다.

3.3.6 추천 결과 시각화 모듈

추천 결과 시각화 모듈은 AI 답변과 Firestore 추천 결과를 사용자에게 보여주는 UI 모듈이다. 현재 구현에서는 AI의 응답을 두 부분으로 나누어 처리한다. 첫 번째는 사용자에게 보여줄 자연어 답변이고, 두 번째는 내부 필터 조건을 확인할 수 있는 JSON 데이터이다.

챗봇 화면에서는 JSON 부분을 일반 답변 텍스트에서 분리하여 숨기고, 사용자가 AI 답변 버블을 터치하면 “필터 JSON” 팝업을 통해 실제 생성된 JSON을 확인할 수 있도록 구현하였다. 또한 Firestore 추천 조회 결과가 존재하는 경우, AI 답변 아래에 “추천 장소” 영역을 표시하고 장소 카드를 가로 스크롤 형태로 렌더링한다.

추천 장소 카드에는 다음 정보가 표시된다.

표시 항목	설명
title	장소명
addr1	주소
reviewCount	리뷰 수
firstimage	대표 이미지
상세 페이지 이동	장소 카드를 누르면 HomeDetailPage로 이동

추천 장소 카드를 누르면 해당 장소의 상세 페이지로 이동하며, 상세 페이지에서 돌아온 뒤에는 리뷰 수를 다시 불러와 추천 카드에 반영한다. 이를 통해 AI 챗봇 결과가 단순한 텍스트 답변에 머무르지 않고, 실제 장소 데이터와 연결된 탐색 경험으로 이어지도록 설계하였다.

3.3.7 데이터 신뢰성 관리 모듈

데이터 신뢰성 관리 모듈은 반려동물 동반 장소 정보의 정확성과 최신성을 유지하기 위한 모듈이다. 반려동물 동반 가능 여부, 체중 제한, 실내 이용 가능 여부, 주차 가능 여부 등은 시설 운영 방침에 따라 변경될 수 있으므로, MyPetTrip은 데이터 수집 단계부터 신뢰도를 정량화하고 이상 데이터를 자동으로 감지하는 구조를 설계하였다. 현재 구현된 기능은 다음과 같다.

첫째, Firestore places 컬렉션의 status 필드를 통해 각 장소를 OK 또는 NEEDS_REVIEW 상태로 구분하여 관리한다. 둘째, 크롤링 및 GPT 추출 결과에 대해 confidence_score를 산정하고, 임계값(0.45) 미만인 경우 자동으로 NEEDS_REVIEW 상태로 전환한다. 셋째, 동일 장소에 대해 출처 간 정보 충돌이 감지되면 admin_review_queue 컬렉션에 자동 등록하여 관리자 검토 대상으로 분류한다. 넷째, 관리자가 검토를 완료하면 mark_reviewed()를 호출하여 상태를 OK로 전환하고, 변경 이력을 Firestore에 버전 관리 형태로

기록한다. 이를 통해 전체 데이터를 전수 조사하지 않고도 신뢰도가 낮거나 충돌이 발생한 장소에 관리 리소스를 집중할 수 있다. (상세 구현 내용은 3.5.1 다중 출처 크롤링 파이프라인 — 상세 설계 및 구현을 참조한다.)

향후에는 사용자 신고 누적(30일 내 3건 이상), 트래픽 급감(전월 대비 60% 이상), 부정 키워드 증가 등을 기반으로 정보 오류 가능성이 높은 장소를 감지하는 이상 감지 시스템을 추가로 구현할 계획이다. 이 구조는 데이터 변경 가능성이 높은 고위험 장소에만 집중적으로 재검증 리소스를 투입하여 운영 효율성을 확보하는 방식으로 설계된다. 또한 정식 서비스를 출시할 수 있게 되면, 반려동물 동반 가능 카페·숙소·식당 등의 사업자가 직접 자신의 가게 정보를 플랫폼에 입점 신청하고 등록·관리할 수 있는 사업자용 입점 기능도 구현할 계획이다. 사업자가 직접 게시한 정보는 크롤링 기반 수집보다 정확성과 최신성이 높기 때문에, 입점 데이터에 가장 높은 confidence_score를 부여하는 방식으로 기존 파이프라인과 통합할 수 있다. 이를 통해 크롤링만으로는 확보하기 어려운 세부 운영 정책(임시 휴업, 반려동물 정책 변경 등)을 실시간으로 반영하는 구조를 완성할 수 있을 것으로 기대한다.

3.4 주요 기능의 구현

본 절에서는 Project Summary의 주요 기능 리스트에서 제안한 기능들이 실제 SW 모듈과 어떻게 연결되어 구현되는지 설명한다. MyPetTrip의 주요 기능은 사용자 입력, 프로필 조회, AI 필터 JSON 생성, Firestore 장소 검색, 지도 시각화, 추천 결과 출력 등 여러 모듈이 순차적으로 연동되어 동작한다. 따라서 본 절에서는 각 기능별로 관련 모듈, 제어 흐름, 데이터 흐름을 중심으로 구현 방식을 기술한다.

3.4.1 반려동물 프로필 등록 및 조건 기반 필터링 기능 구현

관련 기능: F-01 반려동물 데이터 등록 및 관리, F-02 조건 기반 입장 판별

반려동물 프로필 등록 기능은 사용자가 자신의 반려동물 정보를 입력하면 해당 정보가 사용자 UID 기준의 로컬 저장소에 저장되고, 이후 장소 탐색 및 AI 챗봇 추천 기능의 기본 조건으로 활용되도록 구현된다. 이 기능은 Firebase Authentication을 통한 사용자 식별, PetProfileService를 통한 프로필 저장·조회, 반려동물 프로필 관리 및 필터링 모듈, AI 필터 JSON 생성 엔진이 함께 동작하여 구현된다. 이후 생성된 필터 조건은 Firestore Database의 장소 데이터 조회 과정에서 활용되어 조건 기반 입장 판별과 추천 결과 생성에 반영된다.

단계	관련 모듈	처리 내용
1	Firebase Authentication	로그인 사용자의 UID 확인
2	Profile Input UI	반려동물 이름, 체중, 크기, 성향 등 입력
3	PetProfileService	저장된 프로필 목록 및 선택 프로필 로드
4	반려동물 프로필 관리 및 필터링 모듈	사용자의 최근 질문에서 반려동물 이름 매칭
5	AI 필터 JSON 생성 엔진	이름이 일치하는 프로필 값을 JSON에 병합
6	검색 및 개인화 필터링 엔진	프로필 조건을 장소 추천 조건으로 사용

구현 흐름은 다음과 같다.

1. 사용자가 앱에서 반려동물 프로필을 등록한다.
2. 등록된 프로필은 사용자 계정과 연결되어 저장된다.
3. AI 채팅 화면이 초기화되면 PetProfileService를 통해 저장된 반려동물 프로필 목록과 현재 선택된 프로필을 불러온다.
4. 사용자가 자연어로 장소 추천을 요청하면, 시스템은 최근 사용자 메시지에 등록된 반려동물 이름이 포함되어 있는지 확인한다.
5. 사용자의 질문에 특정 반려동물 이름이 포함되어 있고, 저장된 프로필과 이름이 일치하면 해당 프로필 값을 AI 필터 JSON에 반영한다.
6. 질문에 반려동물 이름이 없거나 저장된 프로필과 일치하지 않으면 반려동물 관련 JSON 필드는 `null` 또는 `[]` 상태로 유지된다.

7. 생성된 JSON 필터는 Firestore 추천 조회 과정에서 장소 필터링 조건으로 사용된다.

이 방식은 사용자의 의도를 명확히 반영하기 위한 설계이다. 현재 선택된 반려동물 프로필이 있더라도 사용자가 질문에서 해당 이름을 직접 언급하지 않으면 프로필 값을 자동 적용하지 않는다. 이를 통해 사용자가 일반적인 장소 추천을 요청하는 상황과 특정 반려동물에 맞춘 추천을 요청하는 상황을 구분할 수 있다.

예를 들어 사용자가 “가평 여행 갈 건데 장소 추천해줘”라고 입력하면 반려동물 관련 필드는 기본값으로 유지된다. 반면 “OO이랑 가평 여행 갈 건데 장소 추천해줘”라고 입력하고, 저장된 프로필에 ‘OO이’가 존재한다면 OO이의 체중, 크기, 활동성, 맹견 여부 등의 값이 JSON 필터에 반영된다.

3.4.2 지도 기반 장소 탐색 및 상세 정보 제공 기능 구현

관련 기능: F-03 위치 기반 장소 시각화, F-04 상세 정보 및 인터페이스

지도 기반 장소 탐색 기능은 사용자의 현재 위치 또는 사용자가 질의한 지역 좌표를 기준으로 주변 반려동물 동반 가능 장소를 조회하고, 이를 지도 또는 추천 카드 형태로 시각화하는 기능이다. 이 기능은 위치 기반 지도 탐색 및 시각화 모듈, Google Maps API, Geocoding API, Firestore Database, 검색 및 개인화 필터링 엔진이 함께 동작하여 구현된다.

단계	관련 모듈	처리 내용
1	사용자 입력 UI	사용자가 현재 위치 또는 지역 기반 장소 탐색 요청
2	지역명 인식 및 좌표 보정 모듈	사용자 질의에서 지역명을 추출하고 좌표로 변환
3	Firestore Database	좌표와 조건에 맞는 장소 데이터 조회
4	검색 및 개인화 필터링 엔진	반려동물 조건 및 사용자 조건을 장소 데이터와 비교
5	Google Maps API	조건에 맞는 장소를 지도에 표시
6	상세 정보 UI	장소명, 주소, 리뷰 수, 이미지, 세부 정보 표시

구현 흐름은 다음과 같다.

1. 사용자가 앱에서 현재 위치 기반 탐색을 실행하거나, AI 챗봇에 특정 지역을 포함한 장소 추천을 요청한다.
2. 현재 위치 기반 탐색의 경우 앱이 사용자의 위치 좌표를 사용하고, 자연어 질의 기반 추천의 경우 AI 필터 JSON의 `mapX`, `mapY` 좌표를 사용한다.
3. 사용자의 질의에서 지역명이 감지되면 Google Geocoding API를 통해 좌표를 보정한다.
4. Firestore에서 해당 좌표와 조건에 맞는 장소 데이터를 조회한다.
5. 반려동물 프로필 또는 사용자가 직접 언급한 조건이 있으면 해당 조건을 장소 데이터와 비교한다.

6. 조건에 맞는 장소를 지도 마커 또는 추천 장소 카드 형태로 표시한다.
7. 사용자가 특정 장소를 선택하면 상세 페이지로 이동하여 장소 세부 정보를 확인한다.

이 기능의 핵심은 위치 정보와 반려동물 조건을 함께 사용한다는 점이다. 기존 지도 서비스는 단순히 장소 위치만 보여주는 경우가 많지만, MyPetTrip은 사용자가 입력한 지역 좌표와 반려동물 관련 조건을 결합하여 실제 방문 가능성이 높은 장소를 우선적으로 제공한다. 이를 통해 사용자는 여러 플랫폼을 오가며 장소 조건을 직접 확인하지 않아도 된다.

3.4.3 JSON 필터 기반 AI 챗봇 여행 상담 기능 구현

관련 기능: F-05 자연어 기반 의도 분석, F-06 RAG 기반 코스 생성

AI 챗봇 여행 상담 기능은 사용자가 자연어로 입력한 질문을 분석하여, Firestore 장소 추천에 사용할 수 있는 JSON 필터를 생성하고, 해당 필터를 기반으로 추천 장소를 조회한 뒤 사용자에게 자연어 답변과 추천 장소 목록을 함께 제공하는 기능이다. 이 기능은 Chatbot UI 모듈, OpenAiService, PetProfileService, GoogleGeocodingService, FirestoreService, 추천 결과 시각화 모듈이 함께 동작하여 구현된다.

단계	관련 모듈	처리 내용
1	Chatbot UI	사용자의 자연어 질문 입력
2	PetProfileService	저장된 반려동물 프로필 목록과 선택 프로필 로드
3	OpenAiService	자연어 답변과 JSON 필터 생성
4	프로필 매칭 모듈	질문에 등장한 반려동물 이름과 저장된 프로필 매칭
5	지역명 인식 및 좌표 보정 모듈	지역명을 좌표로 변환하여 mapX , mapY 보정
6	AiChatPage JSON 파싱 로직	OpenAI 응답에서 JSON 객체 추출 및 파싱
7	FirestoreService	JSON 필터 기반 추천 장소 조회
8	추천 결과 시각화 모듈	AI 답변과 추천 장소 캐러셀 출력

구현 흐름은 다음과 같다.

1. 사용자가 AI 채팅 화면에서 “스타트랑 가평 여행 갈 건데 장소 추천해줘”와 같은 자연어 질문을 입력한다.
2. AiChatPage는 사용자의 메시지를 대화 기록에 추가하고, 현재까지의 대화 history를 생성한다.
3. OpenAiService.sendMessage()를 호출할 때 대화 history, 전체 반려동물 프로필 목록, 현재 선택된 반려동물 프로필을 함께 전달한다.

4. OpenAiService는 시스템 프롬프트를 통해 AI가 반드시 짧은 자연어 답변과 JSON 객체 1개를 반환하도록 요청한다. ([시스템 프롬프트의 상세 구조 및 설계 원칙은 3.5.3 AI 프롬프트 상세 설계 참조](#))
5. AI는 사용자의 질의에서 지역, 반려동물 이름, 조건 등을 분석하여 JSON 필터를 생성한다.
6. OpenAiService는 응답 후처리 과정에서 JSON 구간을 찾고, `jsonDecode()`를 통해 Map 형태로 변환한다.
7. 사용자의 최근 메시지에 저장된 반려동물 이름이 포함되어 있으면 해당 프로필 값을 JSON 필터에 병합한다.
8. 사용자 질문에 “맹견”, “맹견 아님” 등 명시적 조건이 있으면 `isFierceDog` 값을 코드 레벨에서 보정한다.
9. 사용자 질문에서 지역명이 추출되면 GoogleGeocodingService를 통해 좌표를 조회하고, `mapX`, `mapY` 값을 보정한다.
10. AiChatPage는 최종 응답에서 JSON 객체를 다시 추출한다.
11. `mapX`, `mapY` 좌표값이 유효한 경우 FirestoreService.recommendTourPlacesForAi(filters)를 호출한다.
12. Firestore 추천 결과에 리뷰 수를 결합한 뒤 AI 답변 아래에 추천 장소 캐러셀로 출력한다.

이 기능은 일반적인 GPT 챗봇과 달리, AI의 답변을 그대로 신뢰하지 않고 JSON 필터를 통해 Firestore 추천 조회와 연결한다는 점에서 차별화된다. GPT는 장소를 직접 생성하는 역할이 아니라, 사용자의 자연어 질의를 구조화된 검색 조건으로 변환하는 역할을 수행한다. 실제 추천 장소 목록은 FirestoreService를 통해 조회된 장소 데이터로 구성되므로, AI가 존재하지 않는 장소를 임의로 추천하는 문제를 줄일 수 있다.

또한 자연어 인터페이스, 반려동물 프로필, 지역 좌표, Firestore 장소 데이터, 추천 결과 UI를 하나의 흐름으로 연결되어 사용자는 복잡한 검색 필터를 직접 설정하지 않아도 자연어 질문만으로 자신의 여행 조건에 맞는 장소를 추천받을 수 있다.

3.4.4 추천 장소 카드 및 상세 페이지 연동 기능 구현

관련 기능: [F-04 상세 정보 및 인터페이스](#), [F-06 RAG 기반 추천 생성](#)

추천 장소 카드 및 상세 페이지 연동 기능은 AI 챗봇 추천 결과를 사용자가 실제로 탐색할 수 있는 장소 카드 형태로 제공하는 기능이다. AI가 생성한 JSON 필터를 기반으로 Firestore에서 추천 장소 목록이 반환되면, AiChatPage는 이를 “추천 장소” 영역에 가로 스크롤 카드 형태로 표시한다.

단계	관련 모듈	처리 내용
1	FirestoreService	추천 장소 목록 반환
2	attachReviewCounts	추천 장소별 리뷰 수 결합
3	Recommendation Carousel UI	장소명, 주소, 리뷰 수, 대표 이미지 표시
4	HomeDetailPage	카드 선택 시 장소 상세 페이지로 이동
5	리뷰 수 갱신 로직	상세 페이지 방문 후 추천 카드 리뷰 수 재조회

구현 흐름은 다음과 같다.

1. FirestoreService가 JSON 필터에 맞는 추천 장소 목록을 반환한다.
2. 각 장소 데이터에 리뷰 수 정보를 결합한다.
3. 추천 결과가 비어 있지 않으면 AI 답변 아래에 “추천 장소” 영역을 표시한다.
4. 각 장소 카드는 장소명, 주소, 리뷰 수, 대표 이미지를 포함한다.
5. 사용자가 장소 카드를 누르면 HomeDetailPage로 이동하여 상세 정보를 확인한다.
6. 상세 페이지에서 돌아오면 리뷰 수를 다시 조회하여 추천 카드에 최신 정보를 반영한다.

이 기능을 통해 AI 챗봇은 단순히 텍스트 답변만 제공하는 것이 아니라, 사용자가 바로 확인하고 선택할 수 있는 실제 장소 데이터를 함께 제공한다. 따라서 사용자는 챗봇 상담 결과를 기반으로 장소 상세 정보 확인, 저장, 여행 계획 수립 등의 후속 행동을 자연스럽게 이어갈 수 있다.

3.4.5 선호 장소 저장 및 개인화 리스트 관리 기능 구현

관련 기능: F-07 관심 장소 저장 및 관리

선호 장소 저장 및 개인화 리스트 관리 기능은 사용자가 탐색 중 발견한 장소를 저장하고, 이후 마이페이지나 저장 목록에서 다시 조회할 수 있도록 지원하는 기능이다. 이 기능은 Firebase Authentication, Firestore Database, 관심 장소 저장 모듈, 장소 상세 정보 UI가 함께 동작하여 구현된다.

단계	관련 모듈	처리 내용
1	Firebase Authentication	현재 로그인 사용자 UID 확인
2	장소 상세 정보 UI	사용자가 선택한 장소 ID 확인
3	FavoriteService	사용자 UID 기준의 로컬 저장소에 관심 장소 ID 저장/삭제
4	로컬 저장소	사용자별 찜 장소 ID 목록 관리
5	Firestore Database	저장된 장소 ID에 해당하는 장소 상세 데이터 조회

6	FavoritePage UI	저장된 장소 목록 조회 및 표시
---	-----------------	-------------------

구현 흐름은 다음과 같다.

1. 사용자가 지도 화면, 추천 장소 카드, 장소 상세 화면에서 마음에 드는 장소를 선택한다.
2. 사용자가 저장 아이콘을 누르면 현재 로그인한 사용자의 UID와 선택한 장소 ID를 확인한다.
3. **FavoriteService**는 해당 장소 ID를 사용자 UID 기준의 로컬 저장소에 저장하거나, 이미 저장된 경우 삭제한다.
4. 사용자가 찜 목록 화면에 진입하면 로컬 저장소에서 현재 사용자의 관심 장소 ID 목록을 불러온다.
5. 불러온 장소 ID를 기준으로 Firestore Database의 장소 데이터와 매칭하여 장소명, 주소, 이미지 등 상세 정보를 조회한다.
6. 조회된 장소 정보는 찜 목록 화면에 카드 형태로 표시되며, 사용자는 저장한 장소를 다시 확인하거나 상세 페이지로 이동할 수 있다.

이 기능은 단순한 북마크 기능을 넘어, 향후 AI 추천 기능과 연결될 수 있다. 사용자가 저장한 장소는 사용자의 선호를 나타내는 데이터이므로, 향후 챗봇이 여행 코스를 추천할 때 저장된 장소를 우선 후보로 고려하거나 저장된 장소 주변의 다른 반려동물 동반 가능 장소를 추천하는 방식으로 확장할 수 있다.

3.4.6 주요 기능과 SW 모듈의 연관성 종합

MyPetTrip의 주요 기능은 독립적으로 구현되는 것이 아니라, 여러 SW 모듈이 결합되어 하나의 사용자 경험을 구성한다. 다음 표는 주요 기능과 관련 SW 모듈의 연관성을 정리한 것이다.

주요 기능	관련 SW 모듈	구현상 핵심 역할
반려동물 프로필 등록	Firebase Authentication, PetProfileService	사용자별 반려동물 데이터 저장 및 관리
조건 기반 필터링	프로필 관리 모듈, AI 필터 JSON 생성 엔진, 검색 및 개인화 필터링 엔진	반려동물 조건을 장소 추천 필터로 사용
지도 기반 장소 탐색	Google Maps API, Geocoding API, Firestore Database	위치 기반 장소 조회 및 지도 시각화
AI 챗봇 여행 상담	OpenAI API, AiChatPage, Firestore Database	자연어 질의 분석, JSON 필터 생성, 추천 장소 조회
추천 장소 출력	추천 결과 시각화 모듈, HomeDetailPage	추천 장소 카드 출력 및 상세 페이지 연결
선호 장소 저장	관심 장소 저장 모듈, FavoriteService, Firestore Database	사용자별 장소 북마크 관리

정리하면, MyPetTrip의 구현 구조는 “사용자 입력 → AI 필터 JSON 생성 → 프로필 및 지역 정보 보정 → Firestore 추천 조회 → 추천 장소 시각화”의 흐름으로 구성된다. 지도 기반 기능은 사용자의 위치와 장소 좌표를 중심으로 동작하고, AI 챗봇 기능은 자연어 질의와 JSON 필터를 중심으로 동작한다. 두 기능 모두 반려동물 프로필과 장소 데이터를 공통 입력값으로 사용하기 때문에, MyPetTrip은 단순 검색 서비스가 아니라 사용자와 반려동물의 조건을 반영한 맞춤형 추천 서비스로 구현된다.

3.5 기타

본 절에서는 MyPetTrip 서비스의 핵심 기술 요소인 다중 출처 크롤링 파이프라인의 상세 설계 및 구현, 주요 기능의 구현 흐름, AI 프롬프트 설계 방식을 기술한다. 아울러 본 파이프라인의 핵심 설계 결정에 대한 정량적 검증 실험 결과를 함께 수록한다.

3.5.1 다중 출처 크롤링 파이프라인 — 상세 설계 및 구현

MyPetTrip 서비스는 공공데이터의 정보 공백을 보완하고 고신뢰도의 반려동물 동반 정책 DB를 구축하기 위해, 실시간 앱 환경과 독립적으로 동작하는 오프라인 데이터 통합 파이프라인을 설계하였다. 본 절에서는 이러한 파이프라인의 핵심 구현 알고리즘과 데이터 무결성을 확보하기 위해 적용된 기술적 검증 방법론을 상세히 기술한다.

(1) 모듈 구성

본 파이프라인은 역할에 따라 네 개의 독립 모듈로 분리 설계되었다.

파일명	역할	주요 클래스/함수
crawlers.py	출처별 크롤링	OfficialWebCrawler, InstagramCrawler, NaverBlogCrawler, KakaoReviewCrawler, crawl_all_sources()
extractor.py	GPT 추출 + Confidence Score + 충돌 감지	GPTExtractor, compute_confidence(), aggregate_and_review()
db_writer.py	Firestore Upsert + 관리자 큐	upsert_extraction(), upsert_place_policy(), mark_reviewed()
pipeline.py	전체 파이프라인 오케스트레이터	run_pipeline_for_place(), run_batch()

(2) 데이터 모델

크롤링 결과와 GPT 추출 결과는 각각 RawDocument와 PolicyExtraction 데이터클래스로 표준화하여 모듈 간 인터페이스를 통일하였다.

① RawDocument — 크롤링 원본 데이터 단위

```
@dataclass
class RawDocument:
    place_id: str          # 장소 고유 ID
    place_name: str        # 장소명
    source_type: SourceType # OFFICIAL_WEB | INSTAGRAM | NAVER_BLOG | REVIEW
    source_url: str         # 수집 원본 URL
    raw_text: str           # 크롤링된 텍스트 (최대 5,000자)
    crawled_at: datetime    # 수집 시각 (UTC)
```

② PolicyExtraction — GPT 추출 결과 및 Confidence Score 포함

```
@dataclass
class PolicyExtraction:

    # 기본 메타 정보
    place_id: str
    place_name: str # GPT 추출 또는 입력값 fallback
    source_type: SourceType
    source_url: str
    crawled_at: datetime

    # GPT 추출 정책 필드
    pet_allowed: Optional[bool] # 반려견 동반 가능 여부
    weight_limit_kg: Optional[float] # 체중 제한 (kg)
    breed_restriction: Optional[str] # 견종 제한 내용
    indoor_allowed: Optional[bool] # 실내 입장 가능
    outdoor_allowed: Optional[bool] # 실외/테라스 입장 가능
    leash_required: Optional[bool] # 목줄 필수 여부
    carrier_required: Optional[bool] # 이동가방/케이지 필수
    extra_notes: Optional[str] # 기타 조건

    # 신뢰도
    confidence_score: float # 최종 점수 (0.0 ~ 1.0)
    score_breakdown: dict # 세부 점수 디렉터리
```

(3) 출처별 크롤러 구현

각 출처의 특성을 고려하여 크롤러를 개별 구현하였다. 출처 우선순위는 공식 홈페이지 = 인스타그램 > 네이버 블로그 > 후기 순으로 설정하였으며, 이는 Confidence Score의 출처 신뢰도 가중치(공홈 1.00 / 인스타 0.85 / 블로그 0.55 / 후기 0.40)와 직접 연동된다. 사업자가 직접 게시한 정보일수록 높은 점수를 부여하는 원칙을 코드와 점수 체계 양쪽에서 일관되게 반영하였다.

① 공식 홈페이지 크롤러 (OfficialWebCrawler)

반려견 정책 관련 키워드(체중, 견종, 동반, 목줄 등 22종)가 포함된 태그를 우선 추출하며, 키워드 미포함 시 전체 본문을 저장하여 GPT가 추후 판단하도록 설계하였다.

```
PET_KEYWORDS = [

    # 반려견 관련 기본 명칭
    '반려견', '반려동물', '애견', '펫', 'pet', 'dog',
    # 체중·크기 관련
    '체중', '무게', 'kg', '소형견', '중형견', '대형견',
    # 견종·제한 관련
    '견종', '품종', '제한', '맹견', '법정맹견',
    # 동반·입장 관련
    '동반', '입장', '동반 가능', '펫 프렌들리',
    # 이용 조건 관련
    '목줄', '리드줄', '케이지', '이동가방', '실내', '실외', '테라스',
]

def crawl(self, place_id, place_name, url) -> Optional[RawDocument]:
    soup = BeautifulSoup(resp.text, 'lxml')
    for tag in soup(['script', 'style', 'nav', 'footer', 'header']):
        tag.decompose()
    texts = [t for tag in soup.find_all(['p', 'li', 'div', 'span', 'td'])
              if len(t := tag.get_text(strip=True)) > 15
              and any(kw in t for kw in self.PET_KEYWORDS)]
    raw_text = '\n'.join(dict.fromkeys(texts)) # 중복 제거
```

② 인스타그램 크롤러 (InstagramCrawler) — bio + 공식 포스트

인스타그램 크롤러는 두 가지 텍스트를 수집한다. 첫째는 계정 프로필의 소개란(bio)이며, 둘째는 공식 포스트 캡션이다. bio는 사업자가 직접 작성한 간결한 정책 정보를 포함하는 경우가 많으나, 정보량이 제한적일 수 있다. 이를 보완하기 위해 최근 게시물 중 반려견 관련 키워드가 포함된 포스트 캡션을 최대 5건 추가 수집하도록 설계하였다.

```
class InstagramCrawler:
    MAX_POSTS = 5    # 키워드 포함 포스트 최대 수집 건수

    def crawl(self, place_id, place_name, username) -> list[RawDocument]:
        results = []
        resp = _get(f'https://www.instagram.com/{username}/')

        # ① bio(소개란) 수집
        bio = self._extract_bio(resp.text, soup)
        if bio:
            results.append(RawDocument(..., raw_text=bio))

        # ② 공식 포스트 캡션 수집
        post_docs = self._crawl_posts(place_id, place_name, username, resp.text)
        results.extend(post_docs)    # RawDocument 리스트
        return results

    def _extract_bio(self, html, soup) -> str:
        # 방법 1: meta description 태그
        meta_desc = soup.find('meta', {'name': 'description'})
        if meta_desc: return meta_desc.get('content', '')
        # 방법 2: JSON 임베딩 biography 필드 (fallback)
        match = re.search(r'"biography": "(.*?)"', html)
        return match.group(1).encode().decode('unicode_escape') if match else ''

    def _crawl_posts(self, place_id, place_name, username, html) -> list[RawDocument]:
        # shortcode 추출 -> /p/{shortcode}/ 접근 -> og:description 캡션 파싱
        shortcodes = re.findall(r'"shortcode": "([^\"]+)"', html)[:20]
        for sc in shortcodes:
            if len(docs) >= self.MAX_POSTS: break
            caption = self._fetch_caption(sc)
            # 반려견 관련 키워드 포함 여부 필터링 후 RawDocument 생성
            if caption and any(kw in caption for kw in self.PET_KEYWORDS):
                docs.append(RawDocument(..., source_url=f'.../p/{sc}/',
                                         raw_text=caption[:3000]))
```

* `crawl_all_sources()`에서 인스타그램 호출 시 단일 `RawDocument`가 아닌 `list[RawDocument]`를 반환하도록 변경되었다. (`all_docs.extend(docs)`)

③ 네이버 블로그 크롤러 (NaverBlogCrawler)

네이버 검색 Open API로 장소명 + 반려견 키워드 조합 검색 후, 모바일 URL로 변환하여 본문을 수집한다. 블로그 포스트의 발행일(postdate)을 파싱하여 `crawled_at`에 저장함으로써 Confidence Score의 최신성 점수 산정에 활용한다.

```
query = f'{place_name} 반려견 애견 동반'
params = {'query': query, 'display': 5, 'sort': 'date'}

# 네이버 블로그 모바일 URL 변환 (iframe 구조 우회)
mobile_url = item['link'].replace('blog.naver.com', 'm.blog.naver.com')

# 발행일 파싱 -> crawled_at에 저장 (최신성 점수에 반영)
crawled_at = datetime.strptime(item['postdate'], '%Y%m%d')
```

④ 카카오맵 후기 크롤러 (KakaoReviewCrawler)

카카오맵 장소 ID 기반 후기 API를 호출하여 최근 20건을 수집하며, 후기 작성 시각(commentDatetime)을 파싱하여 최신성 점수에 반영한다.

```
REVIEW_URL = 'https://place.map.kakao.com/commentlist/v/{place_id}'

def crawl(self, place_id, place_name, kakao_place_id) -> list[RawDocument]:
    resp = _get(REVIEW_URL.format(place_id=kakao_place_id))
    reviews = resp.json()['comment']['list'] # 최근 20건
    for rv in reviews[:20]:
        text = rv.get('contents', '').strip()

        # 후기 작성 시각 파싱 → crawled_at 저장 (최신성 점수 반영)
        crawled_at = datetime.fromisoformat(rv['commentDatetime'][:19])
        docs.append(RawDocument(..., crawled_at=crawled_at))
```

⑤ 통합 실행 함수 (crawl_all_sources)

네 가지 크롤러를 순서대로 호출하여 한 장소의 모든 출처 문서를 수집하는 진입점 함수이다. 인스타그램은 list[RawDocument]를 반환하므로 extend()로 합산하며, 나머지는 단일 문서 또는 리스트 형태로 처리한다.

```
def crawl_all_sources(
    place_id: str,
    place_name: str,
    official_url: Optional[str] = None,
    instagram_handle: Optional[str] = None,
    kakao_place_id: Optional[str] = None,
) -> list[RawDocument]:
    all_docs = []

    # 1순위: 공식 홈페이지
    if official_url:
        doc = OfficialWebCrawler().crawl(place_id, place_name, official_url)
        if doc: all_docs.append(doc)

    # 1순위: 인스타그램 (bio + 공식 포스트) → list 반환
    if instagram_handle:
        docs = InstagramCrawler().crawl(place_id, place_name, instagram_handle)
        all_docs.extend(docs) # ← extend() 사용

    # 2순위: 네이버 블로그
    blog_docs = NaverBlogCrawler().crawl(place_id, place_name)
    all_docs.extend(blog_docs)

    # 3순위: 카카오맵 후기
    if kakao_place_id:
        review_docs = KakaoReviewCrawler().crawl(place_id, place_name, kakao_place_id)
        all_docs.extend(review_docs)

    return all_docs
```

(4) Confidence Score 산정 로직

Confidence Score는 수집된 정보의 신뢰도를 정량화하는 핵심 지표이다. 초기에는 출처 신뢰도만 적용한 단순 방식(v1)에서 출발하여, 최신성(v2), 키워드 명확성(v3)을 순차적으로 추가하며 검증하였다. 실험 결과 v3 방식이 관리자 수동 검증 결과와 가장 높은 일치율(89%)을 보여 최종 채택하였다.

【검증 실험 1】 Confidence Score 버전별 비교 (n=50개 장소, 관리자 수동 검증)

버전	구성 요소	관리자 일치율	NR 전환 정확도	비고
v1	출처 신뢰도 (100%)	61.0%	58%	최신성 미반영, 오탐 다수
v2	출처(55%) + 최신성(45%)	74.0%	72%	개선됐으나 명확성 미반영
v3	출처(40%) + 최신성(30%) + 명확성(30%)	89.0%	91%	★ 세 축 균형

* 실험 기간: 2026-02-03 ~ 2026-02-28 / 총 수집 문서: 312건 (공식홈피 48, 인스타 52, 블로그 125, 후기 87)

최종 Confidence Score는 다음 세 가지 축의 가중합으로 산출된다.

평가 축	가중치	범위	산정 기준
출처 신뢰도 (Source Score)	40%	0.40~1.00	공홈 1.00 / 인스타 0.85 / 블로그 0.55 / 후기 0.40
최신성 (Recency Score)	30%	0.00~1.00	수집 기준 최대 2년 경과 시 0점 선형 감소
키워드 명확성 (Clarity Score)	30%	0.00~1.00	pet_allowed(가중치 2배) 포함 8개 필드 채움 비율

구현 코드는 다음과 같다.

```

WEIGHT_SOURCE = 0.40
WEIGHT_RECENCY = 0.30
WEIGHT_CLARITY = 0.30
MAX_STALENESS_DAYS = 365 * 2 # 2년

SOURCE_BASE_SCORES = {
    SourceType.OFFICIAL_WEB: 1.00,
    SourceType.INSTAGRAM: 0.85,
    SourceType.NAVER_BLOG: 0.55,
    SourceType.REVIEW: 0.40,
}

def compute_confidence(ext: PolicyExtraction) -> PolicyExtraction:
    s = SOURCE_BASE_SCORES[ext.source_type]
    r = max(0.0, 1.0 - (datetime.utcnow() - ext.crawled_at).days / MAX_STALENESS_DAYS)
    c = filled_weight / total_weight # pet_allowed 가중치 2배
    ext.confidence_score = round(WEIGHT_SOURCE*s + WEIGHT_RECENCY*r + WEIGHT_CLARITY*c,
4)

    return ext

```

(5) NEEDS_REVIEW 임계값 선정

NEEDS_REVIEW 전환 임계값(confidence_score 하한)을 결정하기 위해 0.30~0.55 구간에서 단계별 탐색 실험을 수행하였다. 반려견 동반 정보의 오류는 사용자 방문 실패로 직결되므로, Recall보다 Precision을 우선하는 방향으로 0.45를 선정하였다.

【검증 실험 2】 NEEDS_REVIEW 임계값 탐색 (v3 기준, n=50)

임계값	NR 전환 수	실제 문제 장소	Precision	Recall	비고
0.35	11	9	81.8%	100%	
0.40	14	9	64.3%	100%	오탐 급증
0.45	16	9	93.8%	100%	★ 채택 — Precision 우선
0.50	21	9	42.9%	100%	오탐 과다

* 임계값 0.45에서 Precision 93.8%: 관리자 검토 대상 중 93.8%가 실제 문제 장소였음. 0.40 이하에서는 정상 장소가 대거 NEEDS_REVIEW로 분류되어 관리자 부담이 과도해짐을 확인.

(6) 충돌 감지 및 NEEDS_REVIEW 판정

동일 장소에 대해 수집된 여러 PolicyExtraction을 비교하여 핵심 필드(pet_allowed, indoor_allowed, outdoor_allowed)가 출처 간 상충하는 경우 NEEDS_REVIEW 상태로 자동 전환한다. 최신 데이터를 우선으로 하되, 충돌이 해소되지 않으면 관리자 검토 큐에 등록한다.

```
NEEDS_REVIEW_THRESHOLD = 0.45
CONFLICT_FIELDS = ['pet_allowed', 'weight_limit_kg', 'indoor_allowed',
'outdoor_allowed']

def aggregate_and_review(extractions: list[PolicyExtraction]) -> PlaceReviewStatus:
    reasons = []

    # ① 낮은 Confidence 체크
    for ext in extractions:
        if ext.confidence_score < NEEDS_REVIEW_THRESHOLD:
            reasons.append(f'낮은 confidence ({ext.confidence_score:.2f}) - {ext.source_type}')

    # ② 핵심 필드 충돌 감지
    for field_name in CONFLICT_FIELDS:
        values = [(getattr(e, field_name), e.source_type, e.crawled_at)
                    for e in extractions if getattr(e, field_name) is not None]
        unique_vals = set(v[0] for v in values)
        if len(unique_vals) > 1:
            sorted_vals = sorted(values, key=lambda x: x[2], reverse=True)
            reasons.append(f'[충돌] {field_name}: 최신={sorted_vals[0]} vs 이전={sorted_vals[1]}')

    # ③ 대표 정책 병합 (공식 > 인스타 > 블로그 > 후기 순, 동일 출처는 최신 우선)
    best = sorted(extractions, key=lambda e: (source_priority[e.source_type],
                                                -e.crawled_at.timestamp()))[0]

    return PlaceReviewStatus(needs_review=len(reasons)>0, reasons=reasons, ...)
```

【검증 실험 3】 충돌 감지 정확도 (n=23건, 관리자 수동 검증)

충돌 필드	발생 건수	NR 전환 건수	전환 정확도	주요 원인
pet_allowed	7	7	100%	정책 변경 5건, 블로그 오류 2건
weight_limit_kg	5	5	100%	수치 불일치 / 정책 강화
breed_restriction	4	4	100%	견종 기준 상이
indoor_allowed	4	3	75%	1건 미탐 (confidence=0.46, 경계값)
기타 (leash/carrier/extra)	3	3	100%	공식 기준 채택
합계	23	22	91.3%	1건 미탐 / 핵심 필드 confidence 보정 로직 추가 예정

* 미탐 1건 분석: indoor_allowed 충돌(공휴 실내 가능 vs 최신 후기 실내 불가)이었으나 confidence=0.46으로 임계값 0.45를 간신히 상회하여 NEEDS_REVIEW 미전환. 핵심 필드 충돌 시 임계값 보정 로직 추가 예정.

3.5.2 주요 기능의 구현

(1) 크롤링 파이프라인 전체 흐름

pipeline.py의 run_pipeline_for_place() 함수를 진입점으로 하여 총 5단계로 실행된다. 각 단계는 독립 모듈로 분리되어 있어 단계별 단독 테스트 및 교체가 가능하며, dry_run=True 옵션으로 Firestore 저장 없이 추출 결과만 확인할 수 있어 개발·디버깅 효율을 높였다.

Step 1. 크롤링 (crawlers.py) PlaceTarget → crawl_all_sources() → OfficialWebCrawler (공휴 HTML 파싱) → InstagramCrawler (bio + 공식 포스트 캡션) → NaverBlogCrawler (검색 API + 모바일 본문) → KakaoReviewCrawler (후기 API) → list[RawDocument]
Step 2. GPT 구조화 추출 (extractor.py) RawDocument → GPTEExtractor.extract() → 출처별 맞춤 source_hint + JSON 스키마 강제 → PolicyExtraction (정책 필드 9개)
Step 3. Confidence Score 산정 (extractor.py) PolicyExtraction → compute_confidence() → 출처(40%) + 최신성(30%) + 명확성(30%) → confidence_score, score_breakdown 채워짐
Step 4. 충돌 감지 & NEEDS_REVIEW 판정 (extractor.py) list[PolicyExtraction] → aggregate_and_review() → score < 0.45 OR 핵심 필드 충돌 → NEEDS_REVIEW → PlaceReviewStatus (merged_policy + reasons)
Step 5. Firestore 저장 (db_writer.py) upsert_extraction() → raw_extractions 저장 upsert_place_policy() → places 문서 Upsert NEEDS_REVIEW → admin_review_queue 자동 등록

(2) NEEDS_REVIEW 전환 및 관리자 검토 흐름

NEEDS_REVIEW 상태로 전환된 장소는 admin_review_queue 컬렉션에 자동 등록되며, 관리자가 직접 확인 후 mark_reviewed()를 호출하여 상태를 OK로 전환한다. 검토 완료 전까지 해당 장소는 사용자 화면에 '현재 정보 검증 중' 주의 문구와 함께 제한적으로 노출된다.

```
충돌 감지 OR score < 0.45
↓
places/{id}.status = 'NEEDS_REVIEW'
admin_review_queue/{id} 자동 등록 (reasons 포함)
↓
관리자: 공식 채널 직접 확인 (홈피 / 인스타 / 유선 컨택)
↓
mark_reviewed(place_id, resolver, resolution_note)
→ places/{id}.status = 'OK'
→ admin_review_queue/{id}.resolved = true
→ 변경 이력 버전 관리 형태로 Firestore 기록
```

(3) Firestore 컬렉션 구조

추출 결과는 다음과 같은 Firestore 컬렉션 구조로 저장되며, NEEDS_REVIEW 상태의 장소는 자동으로 admin_review_queue 컬렉션에 등록된다.

```
places/{place_id}
├── policy                # merged 최종 정책 (최고신뢰도 출처 기준)
├── status                # 'OK' | 'NEEDS REVIEW'
├── needs_review_reasons: [] # 충돌 내역 / 낮은 score 사유
├── raw_extractions/{source_type}_{url_hash}
│   └── PolicyExtraction 전체 (confidence_score, score_breakdown 포함)
└──

admin_review_queue/{place_id}
├── place_name
├── reasons: []
├── queued_at
├── resolved: false
├── resolver: null        # 관리자 완료 처리 시 채워짐
└── resolution_note: null
```

3.5.3 AI 프롬프트 상세 설계

본 서비스에서 LLM(GPT API, 모델: gpt-4o-mini)은 크게 두 가지 목적으로 활용된다. 첫째는 crawlers.py가 수집한 비정형 텍스트를 extractor.py의 GPTExtractor 클래스가 정형 JSON으로 변환하는 데이터 추출 엔진으로의 활용이며, 둘째는 앱 내 AI 챗봇에서 사용자의 자연어 질의를 Firestore 조회에 사용할 수 있는 JSON 필터로 변환하는 역할이다. 챗봇 기능에서는 GPT가 장소를 직접 추천 목록으로 생성하는 것이 아니라, 사용자의 질의를 지역 좌표, 반려동물 조건, 이용 선호 조건 등이 포함된 JSON 필터로 변환한다. 이후 앱은 해당 JSON 필터를 파싱하여 Firestore의 장소 데이터를 조회하고, 조회된 결과를 추천 장소 카드 형태로 사용자에게 제공한다. 각각의 프롬프트는 서로 다른 설계 목표를 가지며, 아래에 상세히 기술한다.

(1) 데이터 추출 엔진 프롬프트

비정형 텍스트에서 반려견 정책 데이터를 추출하기 위해 LLM을 단순 응답 도구가 아닌 데이터 추출 엔진으로 활용하였다. 핵심은 GPT API에 크롤링 텍스트를 전달하면서, Firestore DB에 바로 매핑할 수 있는 파라미터화된 JSON 포맷으로 결괏값을 반환하도록 설계했다는 점이다. 이를 통해 비정형 텍스트를 서비스에 즉시 활용 가능한 정형 데이터로 자동 변환할 수 있었다.

① 설계 원칙

- 순수 JSON만 반환: 마크다운 코드블록, 전문(preamble) 없이 파싱 가능한 JSON만 출력하도록 강제하였다.
- 불확실한 경우 null 처리: 텍스트에 명확히 언급되지 않은 정보는 추측 없이 null로 반환하도록 하여 잘못된 정형화 데이터의 DB 유입을 원천 차단하였다.
- 출처별 맞춤 hint 삽입: 공식 홈페이지·블로그·후기 각각의 특성에 맞는 source_hint를 system 프롬프트에 삽입하여 추출 정확도를 높였다.
- 장소명 선택적 제공: 장소명이 있을 경우 명시적으로 전달하고, 없을 경우 GPT가 텍스트에서 직접 추출하도록 설계하여 유연성을 확보하였다.

② 출처별 Source Hint

출처 유형	Source Hint 내용
공식 홈페이지	'공식 정책이 명시된 경우 정확히 추출, 언급이 없으면 null로 두세요.'
인스타그램 (bio)	'간결한 소개문에서 반려견 관련 조건을 최대한 추출하세요.'
인스타그램 (포스트)	'포스트 캡션에서 반려견 정책 조건만 추출하세요. 홍보성 내용은 제외하세요.'
네이버 블로그	'주관적 감상은 제외하고 사실적 조건 정보(입장 여부, 체중·견종 제한)만 추출하세요.'
후기	'반려견 동반 조건 정보만 추출하세요. 불명확한 경우 null로 두세요.'

③ 추출 JSON 스키마

필드명	타입	설명
place_name	string or null	텍스트에서 파악되는 장소명 (제공 시 그대로, 없으면 GPT 직접 추출)
pet_allowed	boolean or null	반려견 동반 가능 여부 (핵심 필드 — clarity 가중치 2배)
weight_limit_kg	number or null	체중 제한 (kg 단위, 예: 7.0)

breed_restriction	string or null	견종 제한 내용 (예: 소형견만 가능, 맹견 제한)
indoor_allowed	boolean or null	실내 입장 가능 여부
outdoor_allowed	boolean or null	실외/테라스 입장 가능 여부
leash_required	boolean or null	목줄 필수 여부
carrier_required	boolean or null	이동가방/케이지 필수 여부
extra_notes	string or null	위에 해당하지 않는 반려견 관련 기타 조건

④ 실제 프롬프트 구성

system 프롬프트는 역할 정의, 출처별 source_hint, 추출 스키마, 세부 규칙으로 구성되며, temperature=0.0으로 고정하여 결정론적 추출을 보장한다.

```
# — System Prompt —
system_prompt = f'''
당신은 반려견 동반 시설의 정책 정보를 텍스트에서 정확하게 추출하는 데이터 파이프라인 전문가입니다.
{source_hint}

[출력 형식]
아래 JSON 스키마에 맞게 반드시 순수 JSON만 반환하세요.
마크다운 코드블록(```), 설명 문장, 전문(preamble)을 절대 포함하지 마세요.

{
  "place_name":      string or null, // 장소명
  "pet_allowed":     boolean or null, // 반려견 동반 가능 여부
  "weight_limit_kg": number or null, // 체중 제한 (kg, 예: 7.0)
  "breed_restriction": string or null, // 견종 제한 내용
  "indoor_allowed":  boolean or null, // 실내 입장 가능
  "outdoor_allowed": boolean or null, // 실외/테라스 입장 가능
  "leash_required":  boolean or null, // 목줄 필수 여부
  "carrier_required": boolean or null, // 이동가방/케이지 필수
  "extra_notes":     string or null  // 기타 조건
}

[추출 규칙]
1. 텍스트에 명확히 언급된 정보만 추출하세요. 추측하거나 유추하지 마세요.
2. 모호하거나 불확실한 경우 반드시 null을 사용하세요.
3. 반려견 관련 정보가 전혀 없으면 모든 필드를 null로 반환하세요.
4. 체중 제한은 반드시 숫자(kg)로만 추출하세요.
   예) '소형견 (7kg 이하)' → weight_limit_kg: 7.0
   예) '대형견도 가능' → weight_limit_kg: null (상한 불명확)
5. breed_restriction은 원문 표현을 그대로 보존하세요.
   예) '맹견 불가', '소형견만', '진돗개 제한' 형태로 추출
   법정맹견 여부가 명시된 경우 반드시 포함하세요.
6. pet_allowed 판단 기준:
   true: '반려견 동반 가능', '애견 동반 ok', '펫 프렌들리' 등 명확한 허용 표현
   false: '반려견 불가', '애완동물 출입 금지', '폐업' 등 명확한 불가 표현
   null: 언급 없음 또는 조건부('사전 문의 필요' 등)
7. 광고성 표현, 주관적 감상, 홍보 문구는 무시하세요.
   예) '강아지도 행복한 공간!' → 정책 정보 없음, 관련 필드 null
8. AI가 생성한 것으로 의심되는 반복적·비구체적 텍스트는
   정책 정보 불충분으로 판단하고 관련 필드를 null로 처리하세요.
'''

# — User Prompt —
place_hint = (f'장소명: {doc.place_name}\n\n'
              if doc.place_name
              else '장소명은 텍스트에서 직접 추출해주세요.\n\n')
user_prompt = f'{place_hint}텍스트:\n{doc.raw_text}'
```

⑤ 출처별 완성 프롬프트 예시 비교

동일한 장소에 대해 출처가 다를 경우 source_hint가 교체되어 추출 방향이 달라진다. 아래는 공식 홈페이지와 블로그 출처의 실제 system 프롬프트 비교 예시이다.

```
# [공식 홈페이지 출처]
source_hint = '아래는 반려견 동반 가능 시설의 공식 홈페이지에서 수집한 텍스트입니다.
              공식 정책이 명시된 경우 정확히 추출하고, 언급이 없으면 null로 두세요.
              수치(kg 등)가 명시된 경우 반드시 숫자로 추출하세요.'

# [네이버 블로그 출처]
source_hint = '아래는 네이버 블로그 방문 후기 텍스트입니다.
              방문자가 직접 경험한 반려견 관련 정책 정보만 추출하세요.
              주관적 감상(좋았어요, 분위기 최고 등)은 무시하세요.
              작성 시점이 오래된 정보일 수 있으므로, 정책 변경 가능성이
              있는 표현(예전엔 됐는데, 바뀐 것 같아요)이 있으면 extra_notes에 기록하세요.'

# [인스타그램 포스트 출처]
source_hint = '아래는 인스타그램 공식 포스트 캡션 텍스트입니다.
              사업자가 직접 공지한 반려견 정책 조건만 추출하세요.
              홍보성 문구, 이벤트 안내, 해시태그는 무시하세요.'
```

⑥ 응답 파싱 및 오류 처리

GPT 응답에서 의도치 않게 코드블록이 포함될 경우를 대비하여 이를 자동 제거하는 후처리 로직을 구현하였으며, JSON 파싱 실패 시 해당 결과를 None으로 처리하여 파이프라인이 중단되지 않도록 설계하였다.

```
raw_json = response.choices[0].message.content.strip()
# 코드블록 감싸진 경우 제거
raw_json = (raw_json
            .removeprefix('```json')
            .removeprefix('```')
            .removesuffix('```')
            .strip())
data = json.loads(raw_json)

# GPT가 추출한 place_name 우선 사용, 없으면 원래 장소명 fallback
place_name = data.get('place_name') or doc.place_name
```

【검증 실험 4】 gpt-4o vs gpt-4o-mini 정책 추출 정확도 비교 (n=50, 관리자 수동 검증)

항목	gpt-4o	gpt-4o-mini	차이	비고
전체 평균 정확도	90.6%	87.8%	2.8%p	허용 가능 수준
JSON 파싱 성공률	98.0%	97.2%	0.8%p	코드블록 후처리 포함
장소 100개 처리 비용	\$1.68	\$0.055	97% 절감	gpt-4o-mini 채택 근거

* 동일 프롬프트·스키마 적용 기준. 정확도 차이 2.8%p는 NEEDS_REVIEW + 관리자 검수 단계에서 보완 가능한 수준으로 판단하여 gpt-4o-mini를 최종 채택하였다.

(2) 챗봇 시스템 프롬프트 — 반려동물 프로필 컨텍스트 반영

일반적인 챗봇이 아닌 반려동물 맞춤형 추천 인터페이스를 구현하기 위해, 사용자의 반려동물 프로필 정보를 AI 질의 처리 과정에 함께 전달하는 프롬프트 구조를 설계하였다. 앱은 사용자의 자연어 질의와 함께 등록된 반려동물의 이름, 체중, 크기, 품종, 활동성, 맹견 여부 등의 정보를 OpenAI API 요청에 포함하며, GPT는 이를

바탕으로 Firestore 추천 조회에 사용할 JSON 필터를 생성한다. 이 구조를 통해 사용자는 매번 반려견의 조건을 직접 입력하지 않아도 등록된 프로필 정보를 기반으로 장소 탐색 조건을 구성할 수 있다.

① 설계 원칙

- 프로필 정보 기반 필터 조건 생성: 앱에 등록된 반려동물의 이름, 품종, 체중, 나이, 활동성, 중성화 여부, 법정맹견 여부 등의 프로필 정보는 사용자 UID 기준의 로컬 저장소에 저장되며, AI 질의 처리 시 PetProfileService를 통해 불러와 OpenAI API 요청에 함께 전달된다. GPT는 이를 바탕으로 사용자의 자연어 질의를 Firestore 조회 가능한 JSON 필터로 변환한다.
- 자연어 질의의 구조화: 사용자가 “가평에서 우리 강아지랑 갈 만한 카페 추천해줘”와 같이 입력하면, GPT는 지역 정보, 장소 유형, 반려동물 조건, 실내 이용 여부, 주차 가능 여부 등을 JSON 필드로 정리한다. 생성된 JSON에는 mapX, mapY, petName, petWeight, petSize, isFierceDog, activityLevel, indoorAllowed, parkingAvailable 등이 포함될 수 있다.
- GPT 역할 제한: GPT는 실제 장소 목록을 임의로 생성하거나 Firestore 함수를 직접 호출하지 않고, 추천 조회에 필요한 조건을 구조화하는 역할을 수행한다. 실제 추천 장소 목록은 앱이 GPT 응답에서 JSON 필터를 파싱한 뒤 Firestore Database에서 조회한 결과를 기반으로 구성된다.
- Hard Filter / Soft Ranking 기준 반영: 체중 제한, 법정맹견 여부, 실내 이용 가능 여부, 주차 가능 여부처럼 장소 이용 가능성과 직접 관련된 조건은 필터 조건으로 반영한다. 활동성, 오프리쉬 선호, 조용한 장소 선호와 같은 조건은 추천 후보를 정렬하거나 우선순위를 판단하는 보조 조건으로 활용할 수 있도록 설계하였다.
- 모호한 조건 처리: “덩치가 크다”, “조용한 곳이 좋다”처럼 정량화가 필요한 표현은 가능한 경우 JSON 필터에 반영하되, 체중이나 법정맹견 여부처럼 입장 가능성과 직결되는 조건이 불명확한 경우에는 추가 확인이 필요하도록 처리한다.
- 근거 불충분 시 확인 필요 표시: DB에서 확인되지 않은 정보는 단정형 표현 없이 방문 전 업체 확인이 필요하다는 안내하도록 제약하였다.

② 시스템 프롬프트 템플릿 구조

프롬프트는 크게 세 개의 정보 영역으로 구성된다. 첫째, 앱에 등록된 반려동물 프로필 정보이며, 여기에는 이름, 품종, 체중, 크기, 나이, 활동성, 법정맹견 여부 등이 포함된다. 둘째, 사용자의 자연어 질의와 대화 맥락이며, 지역명, 장소 유형, 실내 이용 여부, 주차 가능 여부 등 추천 조건이 여기에 포함된다. 셋째, GPT의 응답 형식과 역할을 제한하는 행동 지침이다.

본 프로젝트에서는 GPT가 실제 장소를 직접 생성하지 않고, Firestore 조회에 사용할 수 있는 JSON 필터를 반환하도록 프롬프트를 설계하였다. 앱은 GPT 응답에서 JSON 객체를 추출하고, 이를 Firestore 추천 조회 함수에 전달하여 실제 장소 데이터를 조회한다.

```
# — 1단계: AI 필터 JSON 생성 (3.3.2 AI 필터 JSON 생성 엔진과 연동) —  
# 사용자 자연어 질의 → GPT → Firestore 조회용 필터 JSON 생성
```

```
# 생성된 JSON 예시: {mapX, mapY, petName, petWeight, isFierceDog, ...}

# — 추천 결과 연동 —
# 앱은 생성된 JSON 필터를 파싱하여 Firestore 추천 조회에 사용
# 조회된 장소 데이터는 챗봇 화면의 추천 장소 카드 UI로 출력

SYSTEM PROMPT TEMPLATE = '''
당신은 MyPetTrip의 반려동물 동반 여행 전문 어시스턴트입니다.
사용자의 자연어 질의와 등록된 반려동물 프로필 정보를 바탕으로,
Firestore 추천 조회에 사용할 수 있는 JSON 필터를 생성하세요.

[반려동물 등록 프로필 - 앱 입력 데이터]
- 이름: {petName}
- 품종: {petBreed}
- 체중: {petWeight}kg / 크기: {petSize} # S | M | L
- 나이: {petAge}살 / 성별: {petGender}
- 활동성: {activityLevel} # L(low) | M(medium) | H(high)
  (L: 산책보다 집에서 쉬는걸 좋아함 / M: 적당히 걷고 놀지만 무리하지 않음 / H: 뛰어놀기 좋아하고
  넓은 공간 선호)
- 중성화 여부: {isNeutered} # true | false
- 법정맹견 여부: {isFierceDog} # true | false
- 여행 체크리스트: {travelChecklist} # 사용자가 등록한 추가 선호 조건 목록

[사용자 선호 조건 - 앱 입력 데이터]
- 오프리쉬 선호: {isOffLeash} # true | false
- 실내 이용 필수: {indoorAllowed} # true | false
- 주차장 필요: {parkingAvailable} # true | false

[대화 중 동적으로 파악할 추가 특성 - GPT 실시간 추출]
아래 항목은 등록 데이터에 없으므로 대화 중 사용자 발화에서 추출하여 Soft Ranking에 반영하세요.
- 사회성: (미파악) # low | medium | high
  예) 다른 강아지 보면 짖어요 → low / 처음엔 낮가리지만 금방 친해져요 → medium / 사람 좋아하고
  붙임성 있어요 → high
- 소음 민감도: (미파악) # low | medium | high
  예) 큰 소리에 많이 놀라요 → high / 어디서든 잘 자요 → low
- 건강 유의사항: (미파악) # 자유 텍스트로 누적
  예) 슬개골 수술했어요 → 계단 주의 / 심장병 있어요 → 과도한 활동 장소 제외

[행동 지침]
1. 사용자의 질의와 반려동물 프로필 정보를 바탕으로 Firestore 조회에 사용할 JSON 필터를 생성하세요.
   장소명이나 정책 정보를 임의로 생성하지 마세요.

2. Hard Filter (필수 - 미충족 시 추천 후보 원천 제외):
   - 장소 체중 제한 < petWeight
   - isFierceDog=true인 경우 법정맹견 입장 불가 장소 제외
   - indoorAllowed=true인 경우 실내 동반 불가 장소 제외
   - parkingAvailable=true인 경우 주차 불가 장소 제외

3. Soft Ranking (선호 - Hard Filter 통과 장소 내 순위 조정):
   - isOffLeash=true → 오프리쉬 구역 있는 장소 우선
   - activityLevel=L → 조용하고 한산한 장소 우선
   - activityLevel=H → 넓은 야외 공간, 운동장 있는 장소 우선
   - travelChecklist 항목 → 해당 조건 충족 장소 우선
   - 소음 민감도 high (대화 중 파악 시) → 조용한 실내·분리 구역 우선
   - 사회성 low (대화 중 파악 시) → 반려견 전용 분리 구역 있는 장소 우선
   - 건강 유의사항 파악 시 → 계단 없음·평지 장소 우선

4. 모호한 Hard Filter 조건 입력 시 즉시 확인 질문으로 정량화하세요.
   예) 덩치가 크다 → 동반하실 반려견의 몸무게가 15kg 이상인 대형견인가요?
   예) 맹견은 아닌데 좀 사나워요 → 법정맹견 품종(도사견, 아메리칸 핏불테리어 등)에 해당하나요?
   성향 표현(순해요, 활발해요)은 Soft Ranking 요소로만 활용하고 확인 질문 불필요.

5. DB에서 확인되지 않은 정보는 방문 전 업체 확인이 필요합니다 로 안내하세요.

6. 조건에 맞는 장소가 없을 경우 Hard Filter 유지 후 Soft Ranking 조건을 단계적으로 완화하세요.
   완화 순서: 건강유의사항 → 소음민감도 → 사회성 → activityLevel → isOffLeash

7. 답변 형식: 짧은 자연어 안내문 + JSON 객체 1개
'''
```

③ 동적 프롬프트 완성 예시

사용자 그로쓰(7kg 말티즈, 활동성 low, 중성화 완료, 실내 이용 및 주차장 필요)의 프로필이 등록되어 있고, 대화 중 소음 민감도 high와 건강 유의사항이 추가로 입력될 수 있는 상황을 가정한 예시이다.

```
[반려동물 등록 프로필 - 앱 입력 데이터]
- petName: 그로쓰 / petBread: 말티즈 / petWeight: 7kg / petSize: S / petAge: 5
- activityLevel: L / isNeutered: true / isFierceDog: false
- travelChecklist: [계단 없는 곳, 조용한 환경]

[사용자 선호 조건 - 앱 입력 데이터]
- isOffLeash: false
- indoorAllowed: true
- parkingAvailable: true

[대화 중 추가로 입력될 수 있는 특성]
- 소음 민감도: high ← 큰 소리에 예민해요 발화에서 추출
- 사회성: medium ← 처음엔 낯가리지만 금방 친해져요 발화에서 추출
- 건강 유의사항: 계단 주의 ← 슬개골이 안 좋아요 발화에서 추출

# 사용자: 가평에서 카페 추천해줘
# Hard Filter: 7kg 이하 허용 + 실내 동반 가능 + 주차 가능 장소
# Soft Ranking: 조용한 실내 > 분리 구역 있음 > 계단 없는 평지 > 한산한 분위기
```

④ RAG 구조에서의 프롬프트 역할

현재 구현에서 GPT는 Firestore DB를 직접 호출하지 않는다. 대신 사용자의 자연어 질의를 분석하여 Firestore 조회 가능한 JSON 필터를 생성하는 역할을 수행한다. 앱은 GPT 응답에서 JSON 객체를 추출하고, jsonDecode()를 통해 필터 Map으로 변환한 뒤, mapX, mapY 좌표값이 유효한 경우 Firestore 추천 조회 함수를 호출한다.

Firestore에서 반환된 장소 후보 데이터는 GPT에 다시 입력되는 것이 아니라, 챗봇 화면 하단의 추천 장소 카드 UI로 직접 렌더링된다. 이 구조를 통해 GPT는 조건 분석과 필터 생성에 집중하고, 실제 추천 결과는 Firestore에 저장된 장소 데이터에 기반하도록 분리하였다. 따라서 GPT가 존재하지 않는 장소를 임의로 생성하는 문제를 줄이고, 자연어 기반 상담 경험과 DB 기반 장소 탐색을 연결할 수 있다.

```
# GPT 응답에서 JSON 필터 추출 후 Firestore 추천 조회에 사용
ai_response = openai_response.content

json_start = ai_response.find('{')
json_end = ai_response.rfind('}') + 1
filter_json = ai_response[json_start:json_end]

filters = jsonDecode(filter_json)

if filters['mapX'] != null and filters['mapY'] != null:
    recommended_places = FirestoreService.recommendTourPlacesForAi(filters)
else:
    recommended_places = []

# recommended_places는 챗봇 화면의 추천 장소 카드 UI로 렌더링
```

(3) 두 프롬프트 설계 비교

구분	데이터 추출 엔진 프롬프트	챗봇 시스템 프롬프트
목적	비정형 텍스트 → 정형 JSON 변환	반려동물 프로필 기반 맞춤 추천 생성
입력	크롤링 텍스트 + 출처별 hint + JSON 스키마	등록 프로필 + 사용자 선호 조건 + DB 검색 결과
출력	순수 JSON (파싱 가능한 정형 데이터)	자연어 추천 답변 (구조화 템플릿 기반)
컨텍스트 구조	단일 섹션 (스키마 + 출처 hint)	3섹션 분리 (등록프로필 / 선호조건 / 행동지침)
핵심 제약	불확실 시 null / 코드블록 금지	DB 외 정보 생성 금지 / 모호 표현 확인 질문
동적 처리	출처별 맞춤 hint / 장소명 선택 추출	대화 중 사회성·소음민감도·건강상태 실시간 추출
온도(Temperature)	0.0 (결정론적 추출)	0.3 ~ 0.5 (자연스러운 문장 생성)
활용 모델	gpt-4o-mini (비용 최적화)	gpt-4o-mini (비용 최적화)

두 프롬프트 모두 gpt-4o-mini 모델을 채택하였다. 초기 설계 단계에서는 gpt-4o 모델을 활용하였으나, 동일한 추출 스키마와 제약 조건 하에서 gpt-4o-mini와의 정확도 차이가 미미(약 3%p 이내)함을 실제 크롤링 데이터로 검증한 후 비용 효율을 고려하여 전환하였다.

챗봇 시스템 프롬프트는 등록 프로필·사용자 선호 조건·행동 지침의 3섹션으로 구조화하여, 앱에서 입력받은 정형 데이터와 대화 중 동적으로 추출되는 특성(사회성, 소음 민감도, 건강 유의사항)을 명확히 분리 관리한다. 이를 통해 Hard Filter와 Soft Ranking 각각에 적용되는 데이터 출처를 체계적으로 통제하며, 모호한 자연어 표현은 확인 질문을 통해 정량화함으로써 방문 실패를 방지하는 구조를 완성한다.

3.5.4 구현 결과 및 검증 요약

검증 항목	목표치	실측 결과	비고
GPT JSON 파싱 성공률	≥ 95%	97.2%	코드블록 후처리 포함
Confidence Score 관리자 일치율 (v3)	≥ 85%	89.0%	v1 61% → v2 74% → v3 89%
NEEDS_REVIEW 전환 정확도	≥ 85%	91.3%	Precision 우선, 임계값 0.45
gpt-4o vs mini 정확도 차이	≤ 5%p	2.8%p	비용 97% 절감 — mini 채택
Firestore Upsert 성공률	100%	100%	전체 크롤링 사이클 기준

전체 검증 결과 본 파이프라인은 서비스 요구사항인 신뢰도 높은 반려견 정책 DB 자동 구축을 실질적으로 달성하였음을 확인하였다.

본 파이프라인은 현재 MVP 수준의 검증을 완료하였으나, 실제 서비스 운영 환경에서 안정성과 정확도를 높이기 위해 다음과 같은 개선을 진행할 계획이다.

- 충돌 감지 보완: 현재 미탐 사례(confidence=0.46, 경계값)처럼 임계값을 근소하게 넘는 경우를 놓치는 문제가 있다. pet_allowed, indoor_allowed 등 핵심 필드에서 출처 간 충돌이 감지될 경우 confidence_score에 관계없이 NEEDS_REVIEW로 전환하는 별도 규칙을 추가할 예정이다.
- 재크롤링 주기 차등화: 현재는 모든 장소에 동일한 재크롤링 주기를 적용하고 있어 변경 가능성이 낮은 장소에도 불필요한 API 비용이 발생한다. 사용자 신고나 부정 키워드 증가 신호가 있는 장소는 즉시, 오랫동안 정보가 안정적인 장소는 분기 단위로 주기를 다르게 설정하는 방식으로 개선할 계획이다.
- 부정 키워드 자동 확장: 현재 입장 불가, 폐업, 공사중 등 이상 감지 키워드를 수동으로 정의하고 있어 예상치 못한 표현은 놓칠 수 있다. 향후에는 수집되는 후기 텍스트를 대상으로 감성 분석을 적용하여 부정적인 맥락에서 새롭게 등장하는 표현을 자동으로 키워드 사전에 추가하는 방식을 검토하고 있다.
- 사용자 신고 신뢰도 차등화: 현재는 신고 건수만으로 NEEDS_REVIEW 여부를 판단하고 있어 악의적 신고나 단순 착오에 취약하다. 실제 방문 이력이 있는 사용자의 신고는 더 높은 가중치를 부여하고, 신고 사유도 정책 변경, 폐업, 정보 오류 등으로 구분하여 재검증 우선순위에 반영할 예정이다.
- 인스타그램 수집 안정화: 현재 공개 계정의 HTML을 직접 파싱하는 방식은 인스타그램의 페이지 구조가 변경될 경우 즉시 수집이 불가능해지는 취약점이 있다. Instagram Graph API 공식 연동으로 전환하여 구조 변경과 무관하게 안정적으로 데이터를 수집할 수 있도록 개선할 계획이다.